

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Tecnologias para Análise e Desenvolvimento de Sistemas

Trabalho Prático 1 - Etapa 4

Relatório de Testes da API

Sistema de Locadora de Veículos

Aluno: Messias Junio

Professor: Ramon Lacerda Marques

2º semestre de 2026

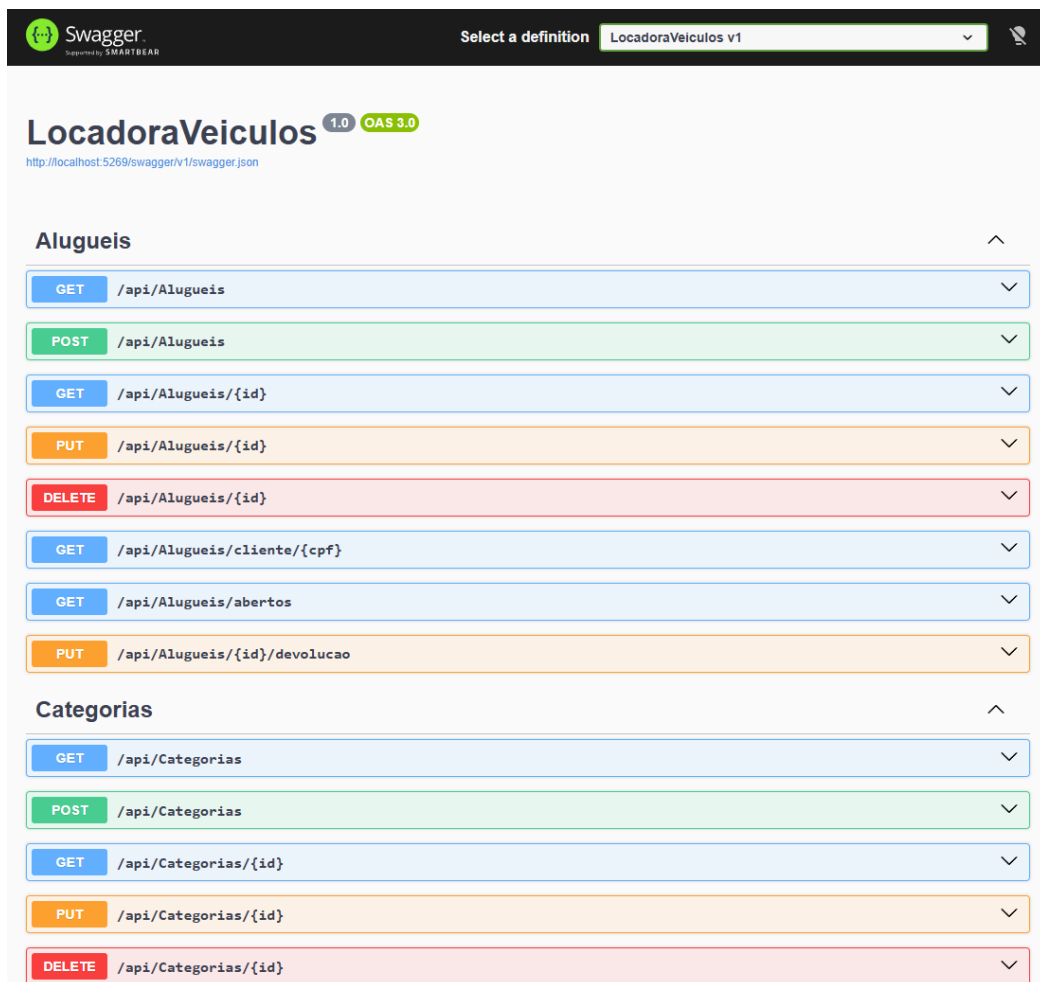
1. Introdução

Este relatório mostra os testes manuais feitos na API da locadora de veículos usando o Swagger, que foi integrado ao projeto e fica disponível em <http://localhost:5269/swagger> quando a aplicação está em execução.

Foram testados todos os métodos de todos os controllers (Fabricantes, Categorias, Veículos, Clientes e Aluguéis), os cinco filtros com join e também situações de erro, para conferir as validações. Para cada teste são apresentados o print da chamada e o print do retorno obtido.

Os testes foram feitos com o banco de dados LocadoraVeiculos no SQL Server Express (localhost\SQLEXPRESS), começando com as tabelas vazias.

Tela inicial do Swagger



Clientes		^
GET	/api/Clientes	▼
POST	/api/Clientes	▼
GET	/api/Clientes/{id}	▼
PUT	/api/Clientes/{id}	▼
DELETE	/api/Clientes/{id}	▼
GET	/api/Clientes/sem-aluguel	▼
Fabricantes		^
GET	/api/Fabricantes	▼
POST	/api/Fabricantes	▼
GET	/api/Fabricantes/{id}	▼
PUT	/api/Fabricantes/{id}	▼
DELETE	/api/Fabricantes/{id}	▼
Veiculos		^
GET	/api/Veiculos	▼
POST	/api/Veiculos	▼
GET	/api/Veiculos/{id}	▼
PUT	/api/Veiculos/{id}	▼
DELETE	/api/Veiculos/{id}	▼
GET	/api/Veiculos/fabricante/{nome}	▼
GET	/api/Veiculos/disponiveis	▼

Figura 1 - Swagger com todos os endpoints da API

2. Fabricantes

Teste 01 - Cadastrar fabricante

Chamada: POST /api/Fabricantes

Resultado esperado: Código 201 com o fabricante criado.

Resultado obtido: Código 201, conforme o esperado.

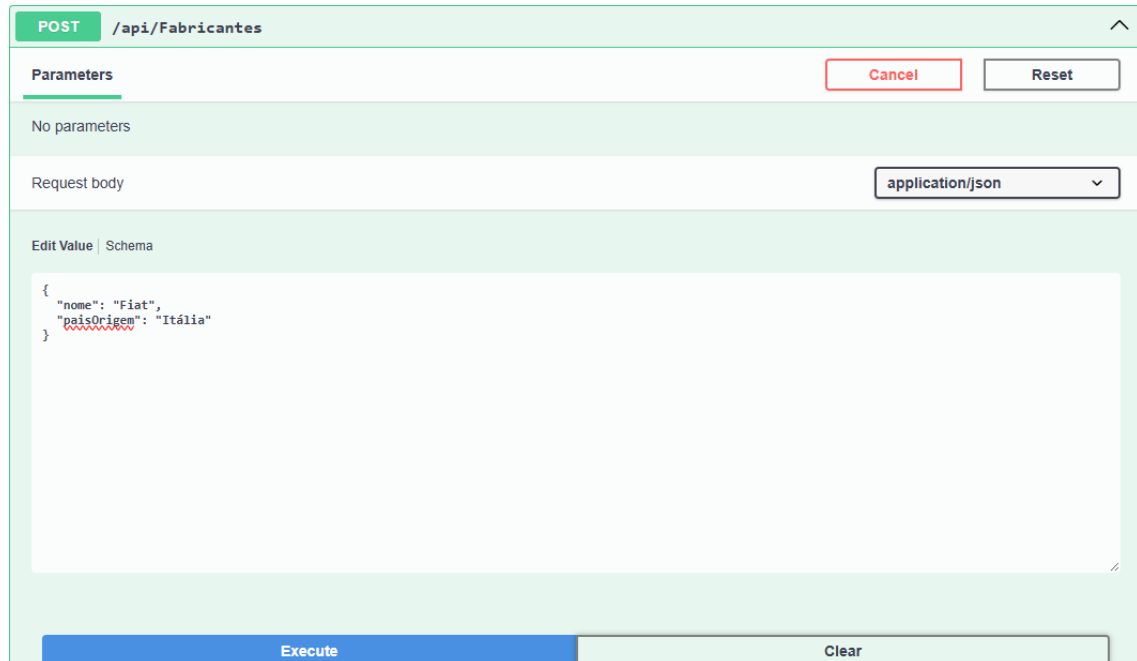


Figura 2 - Chamada do teste 01

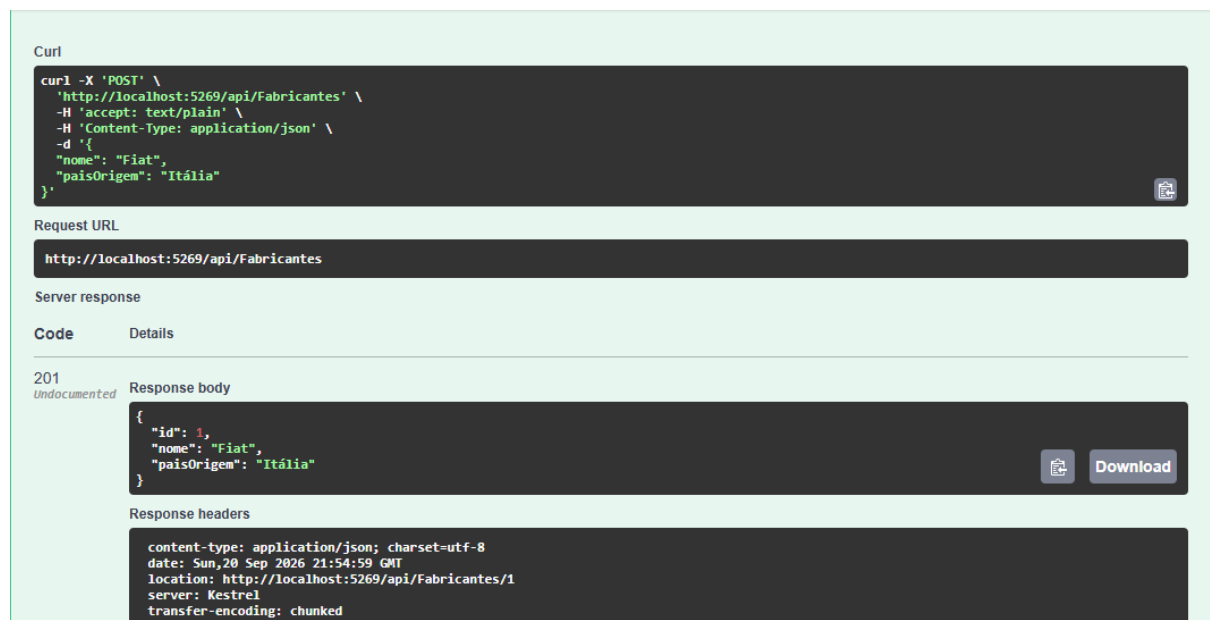


Figura 3 - Retorno do teste 01

Teste 02 - Listar fabricantes

Chamada: GET /api/Fabricantes

Resultado esperado: Código 200 com a lista de fabricantes cadastrados.

Resultado obtido: Código 200, conforme o esperado.

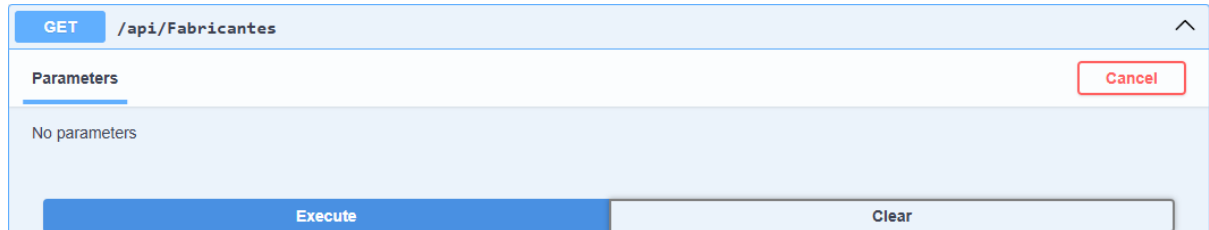


Figura 4 - Chamada do teste 02

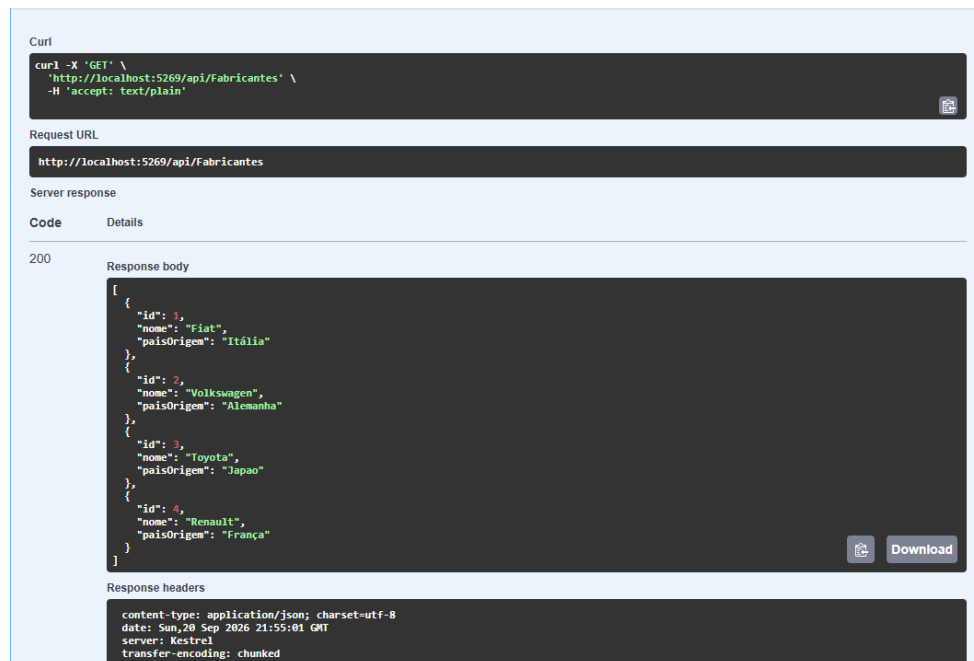


Figura 5 - Retorno do teste 02

Teste 03 - Buscar fabricante por id

Chamada: GET /api/Fabricantes/{id} (id = 1)

Resultado esperado: Código 200 com os dados do fabricante de id 1.

Resultado obtido: Código 200, conforme o esperado.

The screenshot shows a REST client interface with a blue header bar containing 'GET' and the path '/api/Fabricantes/{id}'. Below the header is a 'Parameters' section with a 'Cancel' button. A table with columns 'Name' and 'Description' contains one entry: 'id * required' with a value of '1'. Below the table, the type 'integer(\$int32)' and the role '(path)' are shown. At the bottom, there are 'Execute' and 'Clear' buttons.

Name	Description
id * required	1

integer(\$int32)
(path)

Execute Clear

Figura 6 - Chamada do teste 03

The screenshot shows the response of the GET request. It includes a 'Curl' section with the command: `curl -X 'GET' \ 'http://localhost:5269/api/Fabricantes/1' \ -H 'accept: text/plain'`. The 'Request URL' is `http://localhost:5269/api/Fabricantes/1`. The 'Server response' section shows a 'Code' of 200 and a 'Details' tab. The 'Response body' is a JSON object: `{ "id": 1, "nome": "Fiat", "paisOrigem": "Itália" }`. The 'Response headers' are: `content-type: application/json; charset=utf-8`, `date: Sun, 20 Sep 2026 21:55:04 GMT`, `server: Kestrel`, and `transfer-encoding: chunked`.

Curl

```
curl -X 'GET' \
'http://localhost:5269/api/Fabricantes/1' \
-H 'accept: text/plain'
```

Request URL

```
http://localhost:5269/api/Fabricantes/1
```

Server response

Code 200

Details

Response body

```
{
  "id": 1,
  "nome": "Fiat",
  "paisOrigem": "Itália"
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Sun, 20 Sep 2026 21:55:04 GMT
server: Kestrel
transfer-encoding: chunked
```

Figura 7 - Retorno do teste 03

Teste 04 - Atualizar fabricante

Chamada: PUT /api/Fabricantes/{id} (id = 3)

Resultado esperado: Código 204, fabricante atualizado.

Resultado obtido: Código 204, conforme o esperado.

The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** /api/Fabricantes/{id}
- Parameters:** A table with two columns: Name and Description. The 'id' parameter is listed as 'id * required', 'integer(\$int32)', and '(path)', with a value of '3' entered in the input field.
- Request body:** A dropdown menu is set to 'application/json'.
- Edit Value | Schema:** A text area contains the JSON body:

```
{  "nome": "Toyota",  "paisOrigem": "Japão"}
```
- Buttons:** 'Execute' (blue) and 'Clear' (grey) buttons are at the bottom.

Figura 8 - Chamada do teste 04

The screenshot shows the response of the PUT request in a REST client interface:

- Curl:** A text area contains the curl command:

```
curl -X 'PUT' \  'http://localhost:5269/api/Fabricantes/3' \  -H 'accept: */*' \  -H 'Content-Type: application/json' \  -d '{  "nome": "Toyota",  "paisOrigem": "Japão"  }'
```
- Request URL:** A text field contains 'http://localhost:5269/api/Fabricantes/3'.
- Server response:** A section with 'Code' and 'Details' tabs.
- Code:** The status code '204' is displayed, with 'Undocumented' below it.
- Response headers:** A text area shows the headers:

```
date: Sun,20 Sep 2026 21:55:06 GMT  server: Kestrel
```

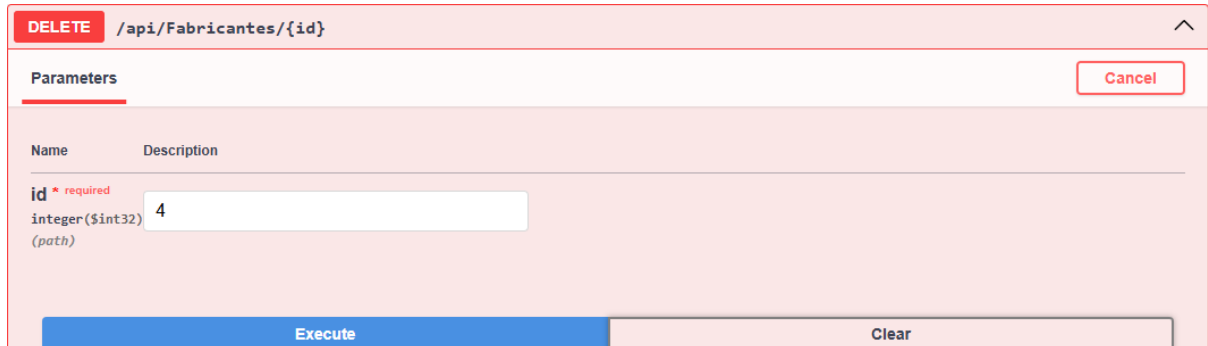
Figura 9 - Retorno do teste 04

Teste 05 - Excluir fabricante

Chamada: DELETE /api/Fabricantes/{id} (id = 4)

Resultado esperado: Código 204, fabricante excluído.

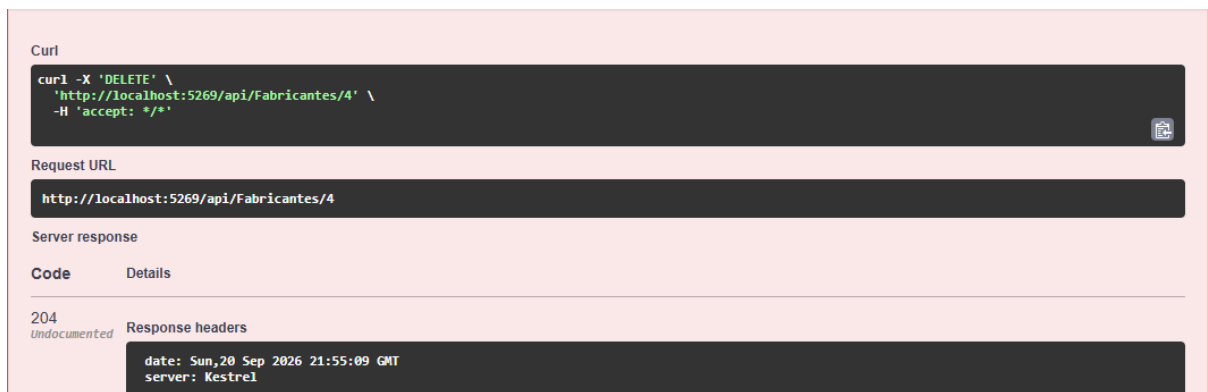
Resultado obtido: Código 204, conforme o esperado.



The screenshot shows a REST client interface with a red header bar. The method is 'DELETE' and the URL is '/api/Fabricantes/{id}'. Below the header, there is a 'Parameters' section with a table. The table has two columns: 'Name' and 'Description'. The first row has 'id' in the 'Name' column, which is marked as 'required' in red. The 'Description' column contains 'integer(\$int32)' and '(path)'. A text input field next to 'id' contains the value '4'. At the bottom, there are two buttons: 'Execute' (blue) and 'Clear' (light gray). A 'Cancel' button is in the top right corner.

Name	Description
id * required	
integer(\$int32)	(path)

Figura 10 - Chamada do teste 05



The screenshot shows the result of a DELETE request. The 'Curl' section displays the command: `curl -X 'DELETE' \ 'http://localhost:5269/api/Fabricantes/4' \ -H 'accept: */*'`. The 'Request URL' section shows 'http://localhost:5269/api/Fabricantes/4'. The 'Server response' section shows a 'Code' of '204' and 'Details' of 'Undocumented'. The 'Response headers' section shows 'date: Sun, 20 Sep 2026 21:55:09 GMT' and 'server: Kestrel'.`

```
curl -X 'DELETE' \
'http://localhost:5269/api/Fabricantes/4' \
-H 'accept: */*'

Request URL
http://localhost:5269/api/Fabricantes/4

Server response
Code    Details
204     Undocumented
Response headers
date: Sun, 20 Sep 2026 21:55:09 GMT
server: Kestrel
```

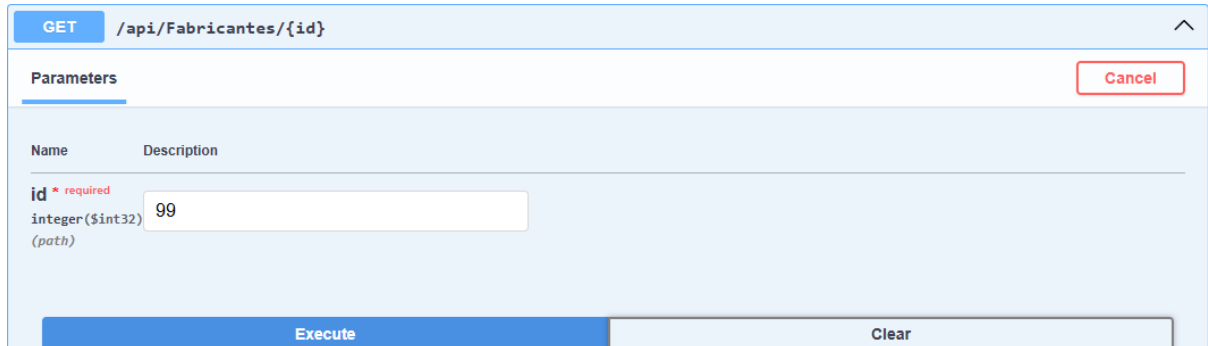
Figura 11 - Retorno do teste 05

Teste 06 - Erro: buscar fabricante que não existe

Chamada: GET /api/Fabricantes/{id} (id = 99)

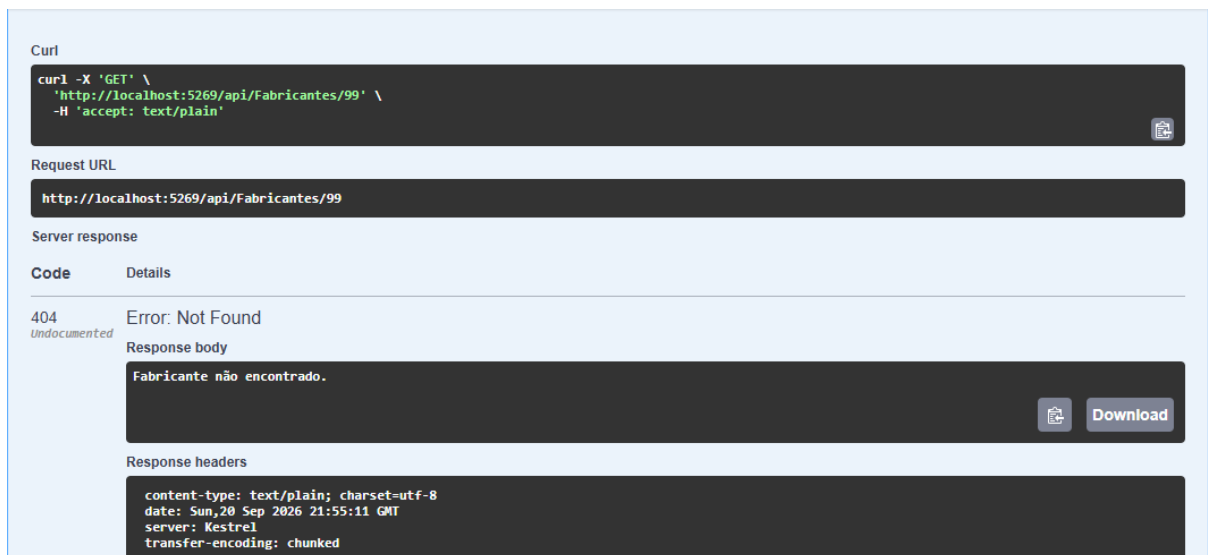
Resultado esperado: Código 404 com a mensagem "Fabricante não encontrado."

Resultado obtido: Código 404, conforme o esperado.



The screenshot shows a REST client interface with a GET request to the endpoint `/api/Fabricantes/{id}`. The `Parameters` tab is active, displaying a table with two columns: `Name` and `Description`. A single parameter is defined: `id` (required, integer(\$int32), path) with a value of `99`. At the bottom, there are `Execute` and `Clear` buttons.

Figura 12 - Chamada do teste 06



The screenshot shows the response of the GET request. The `Curl` tab is active, displaying the command: `curl -X 'GET' \ 'http://localhost:5269/api/Fabricantes/99' \ -H 'accept: text/plain'`. The `Request URL` is `http://localhost:5269/api/Fabricantes/99`. The `Server response` section shows a `404` status code with the message `Error: Not Found`. The `Response body` contains the text `Fabricante não encontrado.`. The `Response headers` section lists: `content-type: text/plain; charset=utf-8`, `date: Sun,20 Sep 2026 21:55:11 GMT`, `server: Kestrel`, and `transfer-encoding: chunked`.

Figura 13 - Retorno do teste 06

3. Categorias

Teste 07 - Cadastrar categoria

Chamada: POST /api/Categorias

Resultado esperado: Código 201 com a categoria criada.

Resultado obtido: Código 201, conforme o esperado.

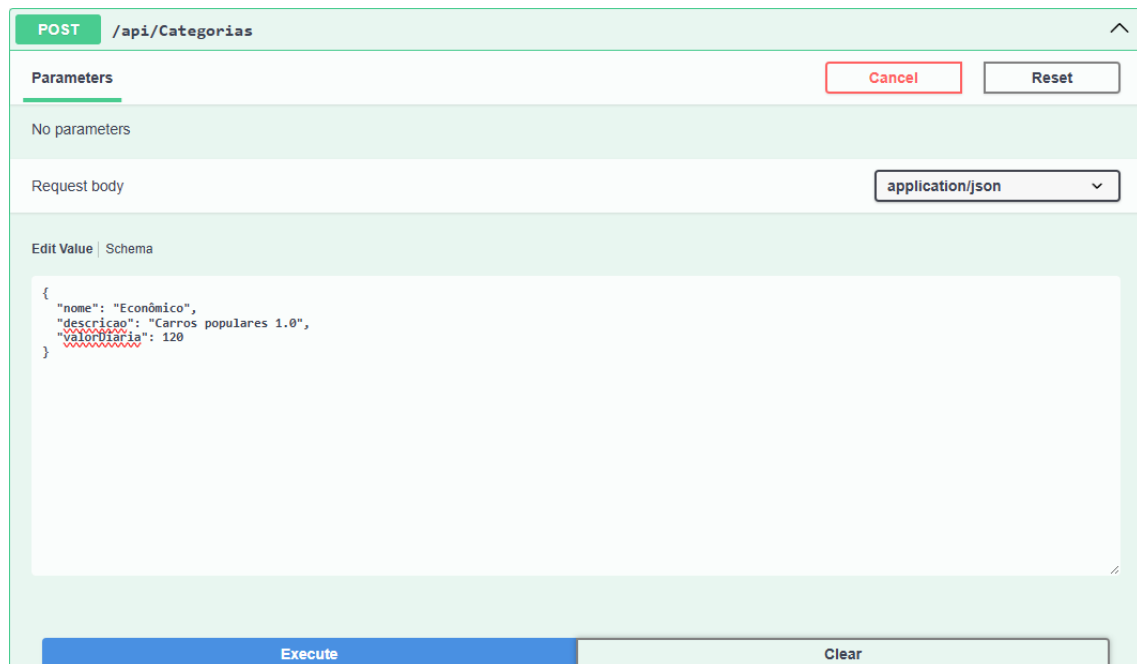


Figura 14 - Chamada do teste 07

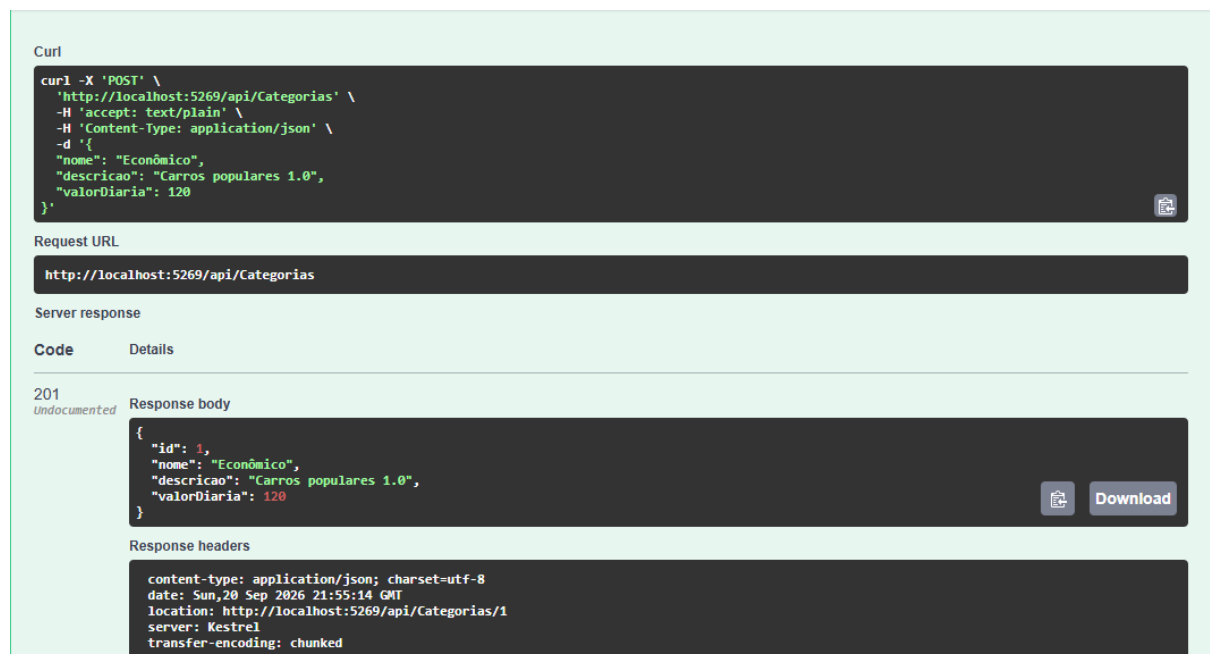


Figura 15 - Retorno do teste 07

Teste 08 - Listar categorias

Chamada: GET /api/Categorias

Resultado esperado: Código 200 com a lista de categorias.

Resultado obtido: Código 200, conforme o esperado.

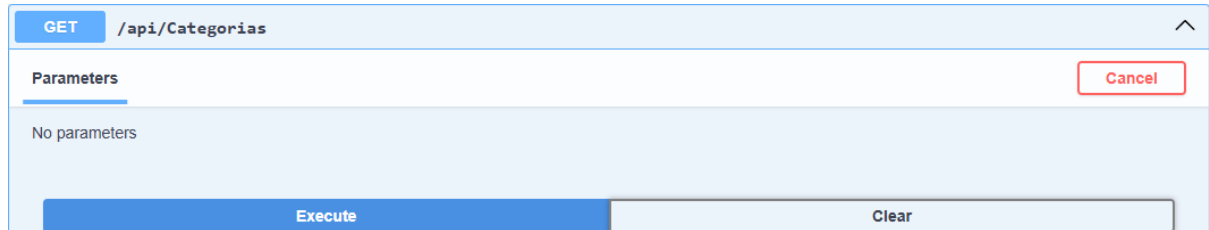


Figura 16 - Chamada do teste 08

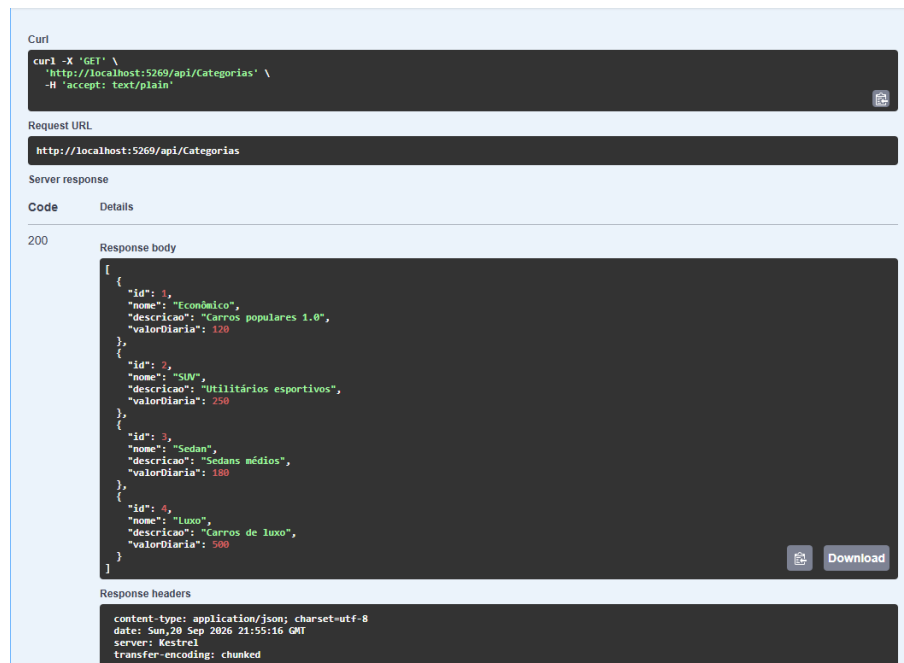


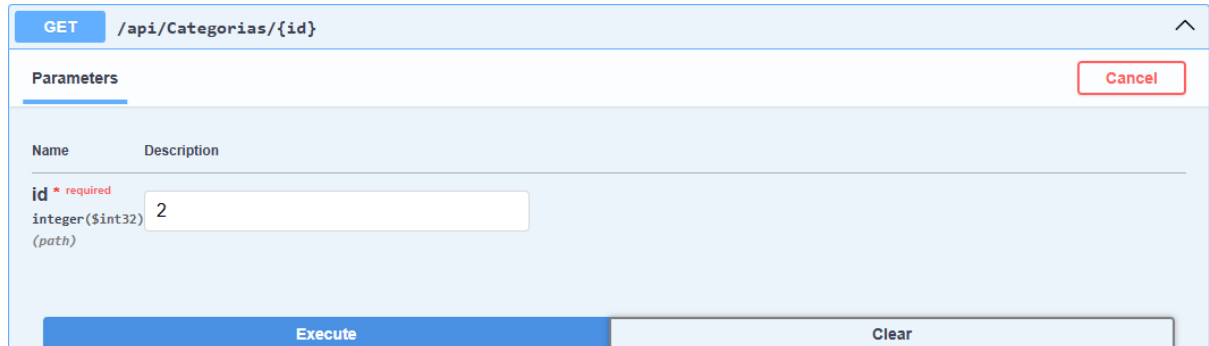
Figura 17 - Retorno do teste 08

Teste 09 - Buscar categoria por id

Chamada: GET /api/Categorias/{id} (id = 2)

Resultado esperado: Código 200 com os dados da categoria de id 2.

Resultado obtido: Código 200, conforme o esperado.



The screenshot shows a REST client interface with a GET request to `/api/Categorias/{id}`. The `Parameters` tab is active, showing a table with two columns: `Name` and `Description`. A single parameter is listed: `id` (required), with a value of `2`. The parameter type is `integer($int32)` and the location is `(path)`. At the bottom, there are `Execute` and `Clear` buttons.

Name	Description
<code>id</code> * required	<code>2</code>

Figura 18 - Chamada do teste 09



The screenshot shows the response of the GET request. The `Request URL` is `http://localhost:5269/api/Categorias/2`. The `Server response` section shows a `Code` of `200` and a `Response body` containing a JSON object: `{ "id": 2, "nome": "SUV", "descricao": "Utilitários esportivos", "valorDiaria": 250 }`. The `Response headers` section shows: `content-type: application/json; charset=utf-8`, `date: Sun, 20 Sep 2026 21:55:18 GMT`, `server: Kestrel`, and `transfer-encoding: chunked`.

```
curl -X 'GET' \
'http://localhost:5269/api/Categorias/2' \
-H 'accept: text/plain'
```

Request URL

`http://localhost:5269/api/Categorias/2`

Server response

Code **Details**

200

Response body

```
{
  "id": 2,
  "nome": "SUV",
  "descricao": "Utilitários esportivos",
  "valorDiaria": 250
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Sun, 20 Sep 2026 21:55:18 GMT
server: Kestrel
transfer-encoding: chunked
```

Figura 19 - Retorno do teste 09

Teste 10 - Atualizar categoria

Chamada: PUT /api/Categorias/{id} (id = 1)

Resultado esperado: Código 204, categoria atualizada (diária passou de 120 para 130).

Resultado obtido: Código 204, conforme o esperado.

The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** /api/Categorias/{id}
- Parameters:** A table with two columns: Name and Description. The first row has 'id' as the Name, marked as 'required', with a value of '1'. Below the input field, it says 'integer(\$int32)' and '(path)'.
- Request body:** A dropdown menu is set to 'application/json'.
- Edit Value | Schema:** A text area containing a JSON object:

```
{  "nome": "Econômico",  "descricao": "Carros populares 1.0",  "valorDiaria": 130}
```
- Buttons:** 'Execute' (blue) and 'Clear' (grey) at the bottom.

Figura 20 - Chamada do teste 10

The screenshot shows the results of the REST client call:

- Curl:** A dark box containing the following command:

```
curl -X 'PUT' \  'http://localhost:5269/api/Categorias/1' \  -H 'accept: */*' \  -H 'Content-Type: application/json' \  -d '{  "nome": "Econômico",  "descricao": "Carros populares 1.0",  "valorDiaria": 130  }'
```
- Request URL:** A dark box containing the URL: `http://localhost:5269/api/Categorias/1`
- Server response:** A section with two tabs: 'Code' and 'Details'.
 - Code:** Shows the status code '204' and the text 'Undocumented'.
 - Details:** A dark box showing response headers: `date: Sun,20 Sep 2026 21:55:21 GMT` and `server: Kestrel`.

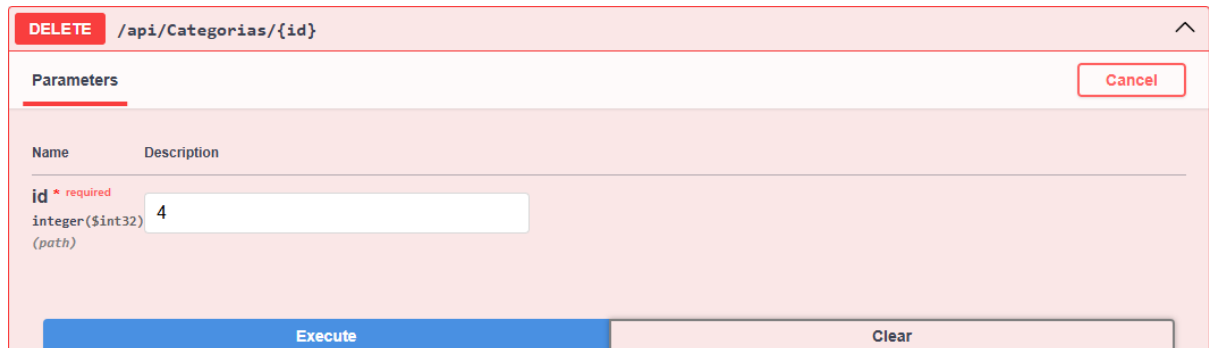
Figura 21 - Retorno do teste 10

Teste 11 - Excluir categoria

Chamada: DELETE /api/Categorias/{id} (id = 4)

Resultado esperado: Código 204, categoria excluída.

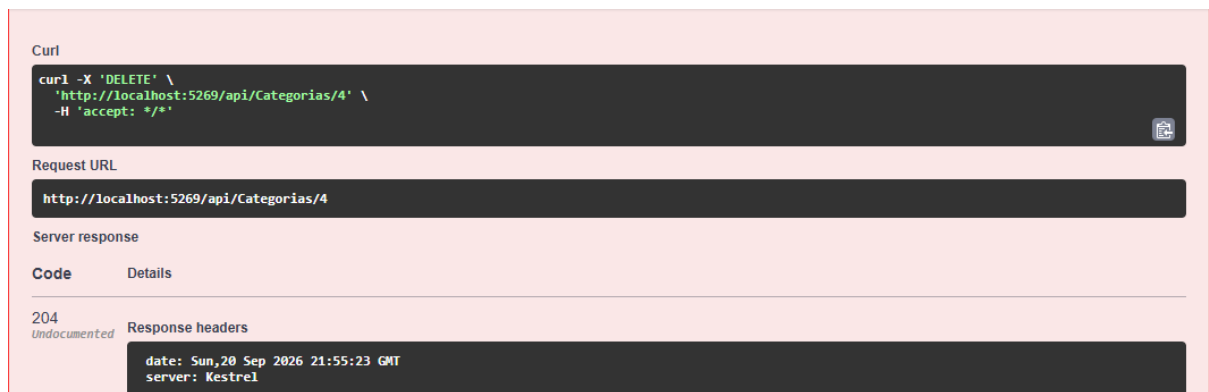
Resultado obtido: Código 204, conforme o esperado.



The screenshot shows a REST client interface with a red header bar. The method is 'DELETE' and the URL is '/api/Categorias/{id}'. Below the header, there is a 'Parameters' section with a table. The table has two columns: 'Name' and 'Description'. The first row has 'id' in the 'Name' column, which is marked as 'required' in red. The 'Description' column contains 'integer(\$int32)' and '(path)'. A text input field next to 'id' contains the value '4'. At the bottom, there are two buttons: 'Execute' (blue) and 'Clear' (light gray). A 'Cancel' button is in the top right corner.

Name	Description
id * required	integer(\$int32) (path)

Figura 22 - Chamada do teste 11



The screenshot shows the response of the DELETE request. The 'Curl' section displays the command: `curl -X 'DELETE' \ 'http://localhost:5269/api/Categorias/4' \ -H 'accept: */*'` . The 'Request URL' section shows 'http://localhost:5269/api/Categorias/4'. The 'Server response' section shows a 'Code' of '204' and 'Details' of 'Undocumented'. The 'Response headers' section shows 'date: Sun, 20 Sep 2026 21:55:23 GMT' and 'server: Kestrel'.

```
curl -X 'DELETE' \
'http://localhost:5269/api/Categorias/4' \
-H 'accept: */*' 
```

Request URL

http://localhost:5269/api/Categorias/4

Server response

Code	Details
204	Undocumented

Response headers

```
date: Sun, 20 Sep 2026 21:55:23 GMT
server: Kestrel
```

Figura 23 - Retorno do teste 11

Teste 12 - Erro: valor da diária inválido

Chamada: POST /api/Categorias

Resultado esperado: Código 400, porque o valor da diária precisa ser maior que zero.

Resultado obtido: Código 400, conforme o esperado.

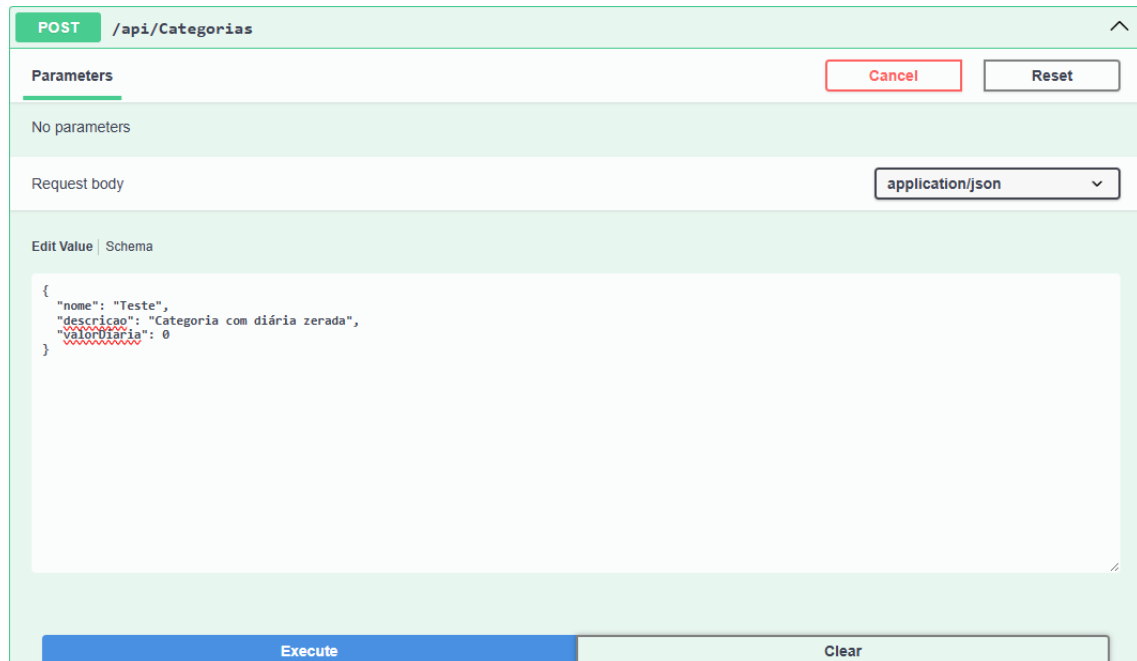


Figura 24 - Chamada do teste 12

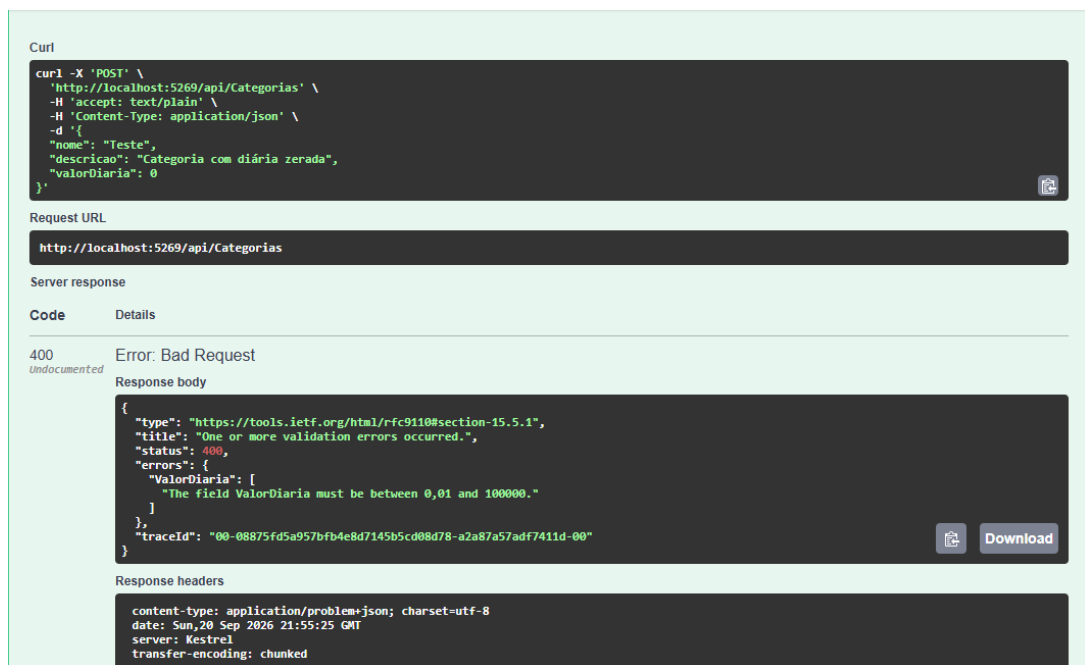


Figura 25 - Retorno do teste 12

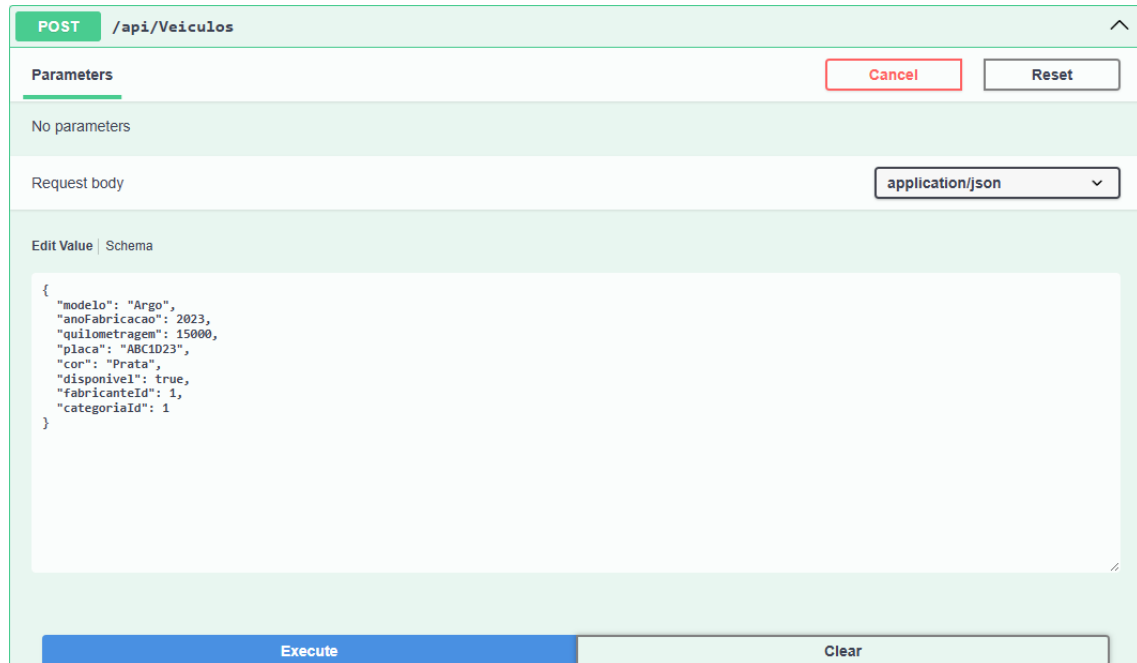
4. Veículos

Teste 13 - Cadastrar veículo

Chamada: POST /api/Veiculos

Resultado esperado: Código 201 com o veículo criado.

Resultado obtido: Código 201, conforme o esperado.

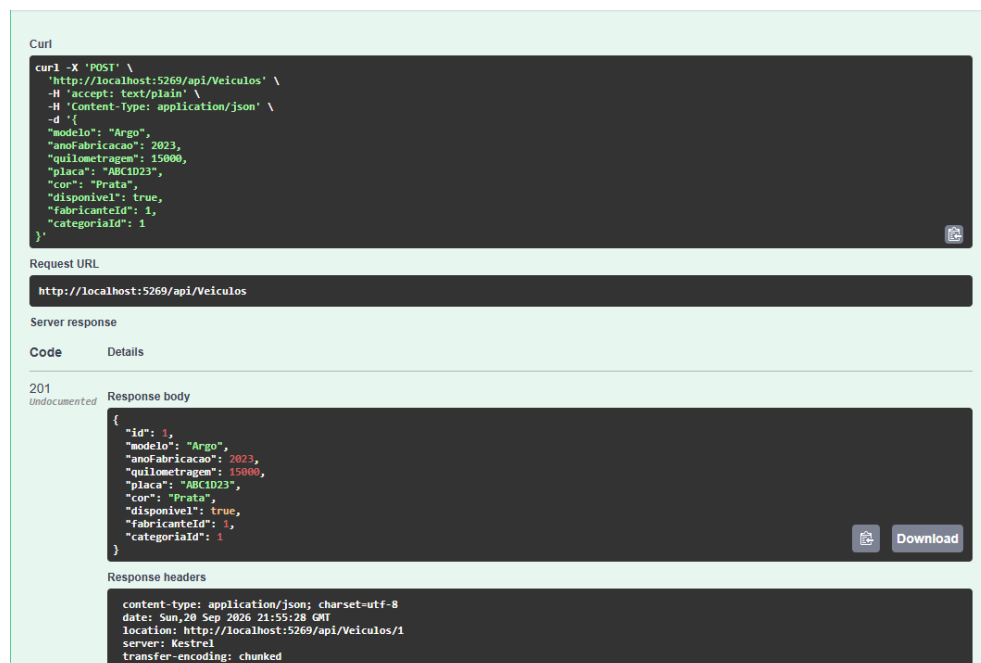


The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** /api/Veiculos
- Parameters:** No parameters
- Request body:** application/json
- Request body content:**

```
{  "modelo": "Argo",  "anoFabricacao": 2023,  "quilometragem": 15000,  "placa": "ABC1023",  "cor": "Prata",  "disponivel": true,  "fabricanteId": 1,  "categoriaId": 1}
```
- Buttons:** Execute, Clear, Cancel, Reset

Figura 26 - Chamada do teste 13



The screenshot shows the response of the POST request to /api/Veiculos. The response is a 201 status code with the following details:

- Request URL:** http://localhost:5269/api/Veiculos
- Server response:** 201
- Response body:**

```
{  "id": 1,  "modelo": "Argo",  "anoFabricacao": 2023,  "quilometragem": 15000,  "placa": "ABC1023",  "cor": "Prata",  "disponivel": true,  "fabricanteId": 1,  "categoriaId": 1}
```
- Response headers:**

```
content-type: application/json; charset=utf-8
date: Sun, 20 Sep 2026 21:55:28 GMT
location: http://localhost:5269/api/Veiculos/1
server: Kestrel
transfer-encoding: chunked
```

Figura 27 - Retorno do teste 13

Teste 14 - Listar veículos

Chamada: GET /api/Veiculos

Resultado esperado: Código 200 com a lista de veículos.

Resultado obtido: Código 200, conforme o esperado.

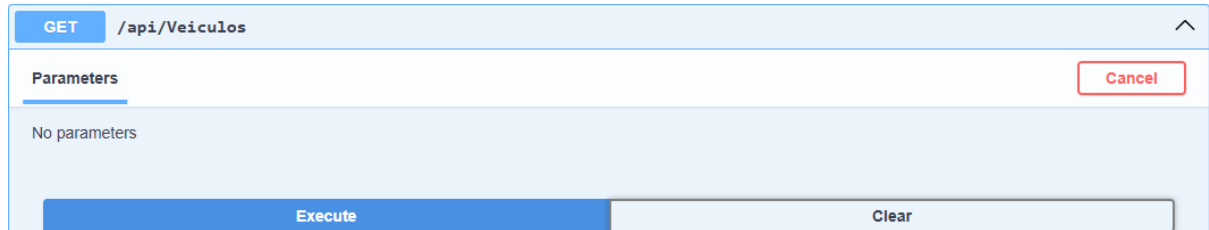


Figura 28 - Chamada do teste 14

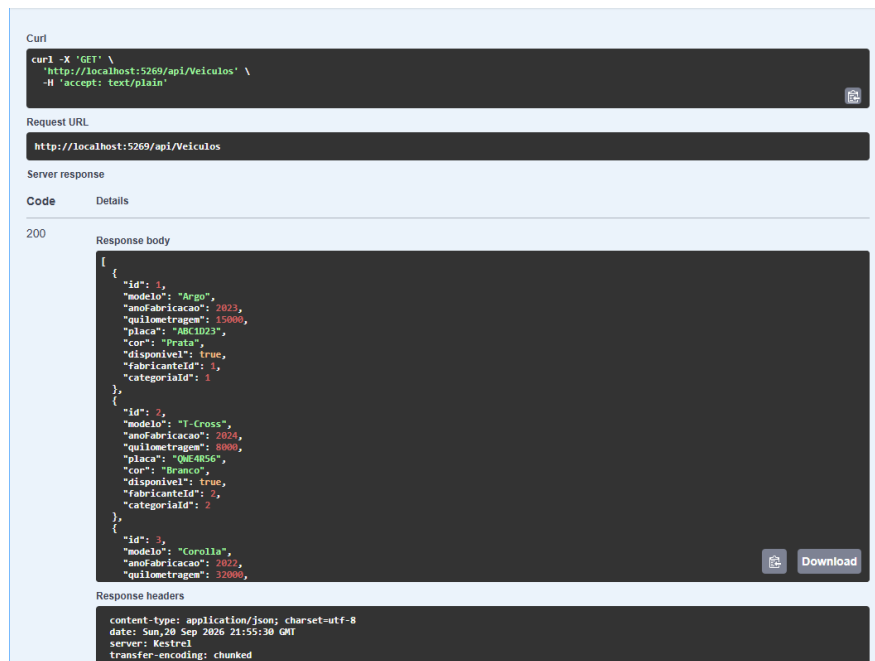


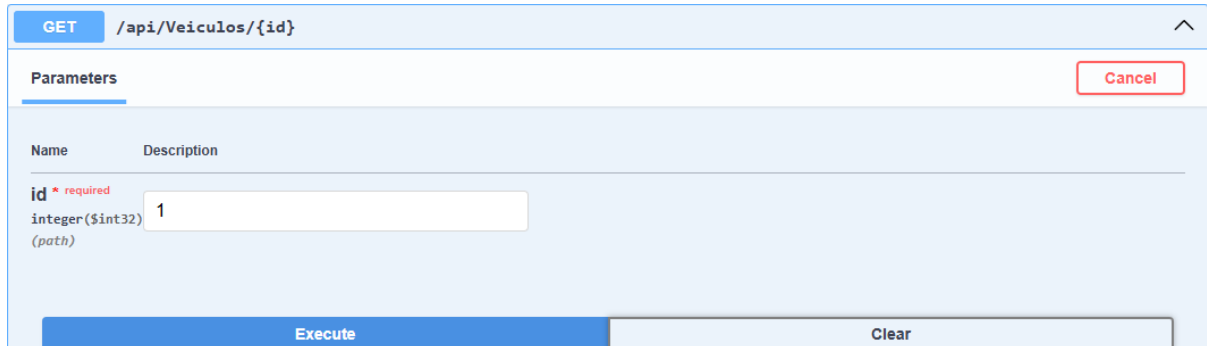
Figura 29 - Retorno do teste 14

Teste 15 - Buscar veículo por id

Chamada: GET /api/Veiculos/{id} (id = 1)

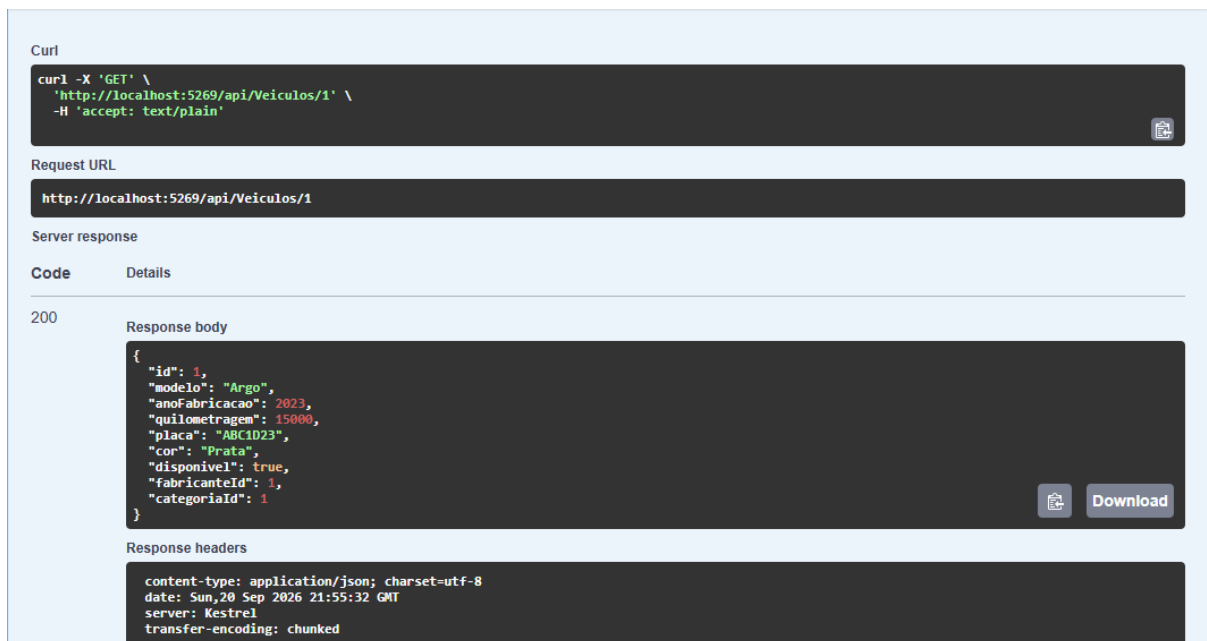
Resultado esperado: Código 200 com os dados do veículo de id 1.

Resultado obtido: Código 200, conforme o esperado.



The screenshot shows a REST client interface with a GET request to `/api/Veiculos/{id}`. The **Parameters** tab is active, displaying a table with two columns: **Name** and **Description**. A single parameter is listed: **id** (marked as required), with a description of `integer($int32)` and `(path)`. The value `1` is entered in the adjacent text field. At the bottom, there are **Execute** and **Clear** buttons. A **Cancel** button is located in the top right corner.

Figura 30 - Chamada do teste 15



The screenshot shows the response of the GET request. The **Request URL** is `http://localhost:5269/api/Veiculos/1`. The **Server response** section shows a **Code** of 200 and a **Response body** containing a JSON object:

```
{
  "id": 1,
  "modelo": "Argo",
  "anoFabricacao": 2023,
  "quilometragem": 15000,
  "placa": "ABC1D23",
  "cor": "Prata",
  "disponivel": true,
  "fabricanteId": 1,
  "categoriaId": 1
}
```

. A **Download** button is next to the response body. The **Response headers** section shows: `content-type: application/json; charset=utf-8`, `date: Sun, 20 Sep 2026 21:55:32 GMT`, `server: Kestrel`, and `transfer-encoding: chunked`.

Figura 31 - Retorno do teste 15

Teste 16 - Atualizar veículo

Chamada: PUT /api/Veiculos/{id} (id = 1)

Resultado esperado: Código 204, veículo atualizado (modelo e cor).

Resultado obtido: Código 204, conforme o esperado.

The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** /api/Veiculos/{id}
- Parameters:** A table with two columns: Name and Description. The first row has 'id' (marked as required) in the Name column and '1' in the Description column. Below the table, the type is specified as 'integer(\$int32)' and '(path)'.
- Request body:** A dropdown menu is set to 'application/json'.
- Edit Value | Schema:** A text area containing a JSON object:

```
{  "modelo": "Argo Drive",  "anoFabricacao": 2023,  "quilometragem": 15000,  "placa": "ABC1D23",  "cor": "Cinza",  "disponivel": true,  "fabricanteId": 1,  "categoriaId": 1}
```
- Buttons:** 'Execute' (blue) and 'Clear' (grey) at the bottom.

Figura 32 - Chamada do teste 16

The screenshot shows the result of the PUT request in a REST client interface:

- Curl:** A code block containing the curl command:

```
curl -X 'PUT' \  'http://localhost:5269/api/Veiculos/1' \  -H 'accept: */*' \  -H 'Content-Type: application/json' \  -d '{  "modelo": "Argo Drive",  "anoFabricacao": 2023,  "quilometragem": 15000,  "placa": "ABC1D23",  "cor": "Cinza",  "disponivel": true,  "fabricanteId": 1,  "categoriaId": 1}'
```
- Request URL:** http://localhost:5269/api/Veiculos/1
- Server response:** A section with a 'Code' column showing '204' and a 'Details' column showing 'Undocumented'.
- Response headers:** A table with two columns: name and value. The first row has 'date' and 'Sun,20 Sep 2026 21:55:35 GMT'. The second row has 'server' and 'Kestrel'.

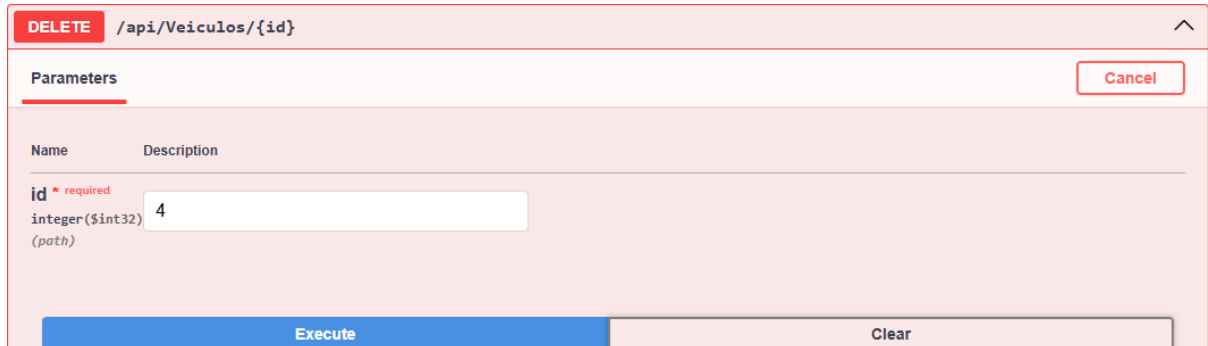
Figura 33 - Retorno do teste 16

Teste 17 - Excluir veículo

Chamada: DELETE /api/Veiculos/{id} (id = 4)

Resultado esperado: Código 204, veículo excluído.

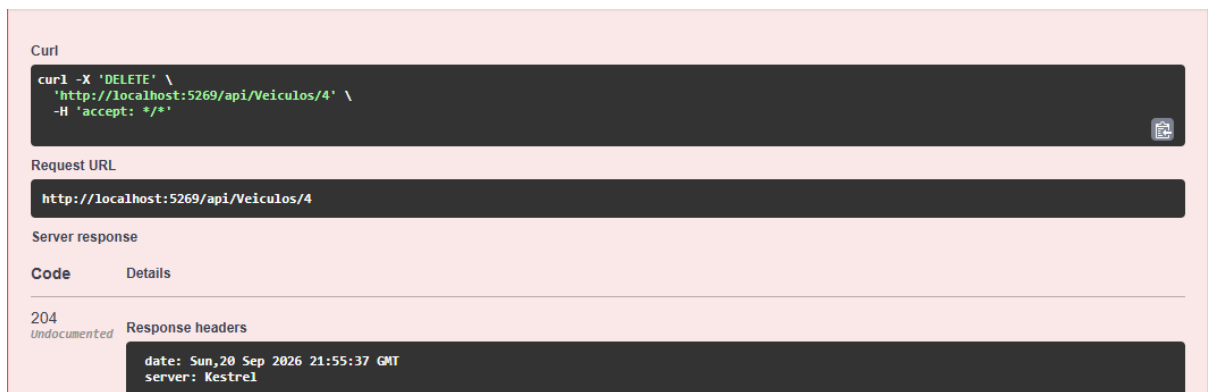
Resultado obtido: Código 204, conforme o esperado.



The screenshot shows an API client interface with a red header bar. The method is 'DELETE' and the URL is '/api/Veiculos/{id}'. Below the header, there is a 'Parameters' section with a 'Cancel' button. A table with two columns, 'Name' and 'Description', lists a parameter 'id' marked as 'required'. The parameter is of type 'integer(\$int32)' and is located in the '(path)' section. A text input field next to 'id' contains the value '4'. At the bottom, there are two buttons: 'Execute' (blue) and 'Clear' (light gray).

Name	Description
id * required	
integer(\$int32)	
(path)	

Figura 34 - Chamada do teste 17



The screenshot shows the same API client interface after the request has been executed. The 'Curl' section displays the command: `curl -X 'DELETE' \ 'http://localhost:5269/api/Veiculos/4' \ -H 'accept: */*'`. The 'Request URL' section shows 'http://localhost:5269/api/Veiculos/4'. The 'Server response' section shows a 'Code' of '204' and 'Details' of 'Undocumented'. The 'Response headers' section shows 'date: Sun, 20 Sep 2026 21:55:37 GMT' and 'server: Kestrel'.

```
curl -X 'DELETE' \
'http://localhost:5269/api/Veiculos/4' \
-H 'accept: */*'

Request URL
http://localhost:5269/api/Veiculos/4

Server response
Code    Details
204     Undocumented
Response headers
date: Sun, 20 Sep 2026 21:55:37 GMT
server: Kestrel
```

Figura 35 - Retorno do teste 17

Teste 18 - Erro: placa já cadastrada

Chamada: POST /api/Veiculos

Resultado esperado: Código 409, porque a placa ABC1D23 já está cadastrada.

Resultado obtido: Código 409, conforme o esperado.

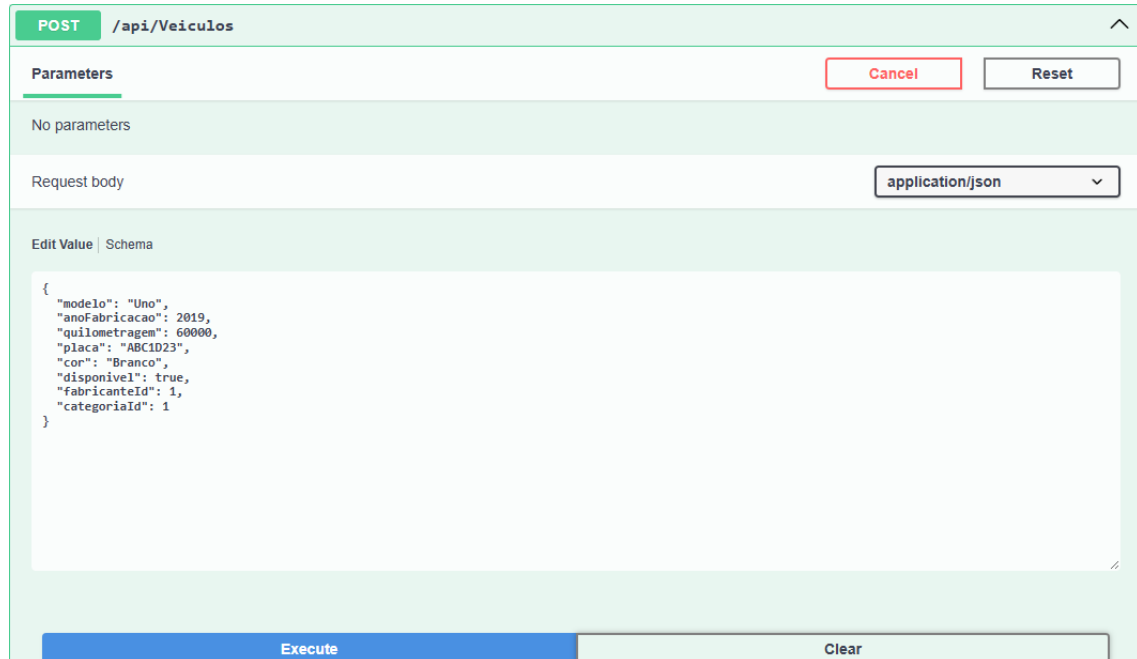


Figura 36 - Chamada do teste 18

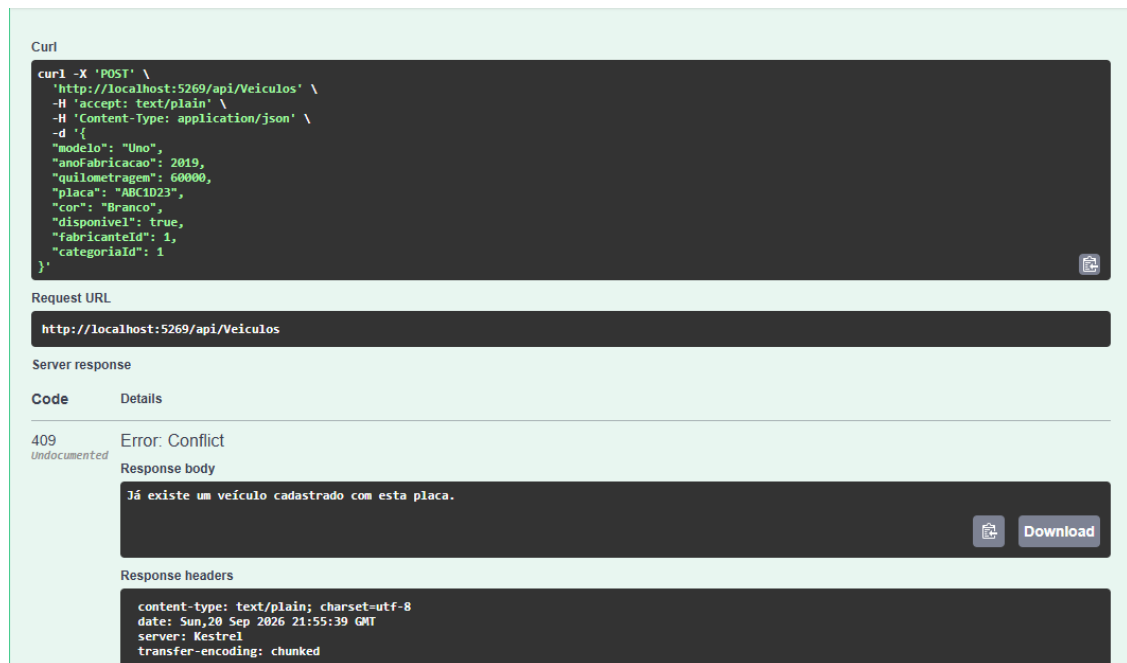


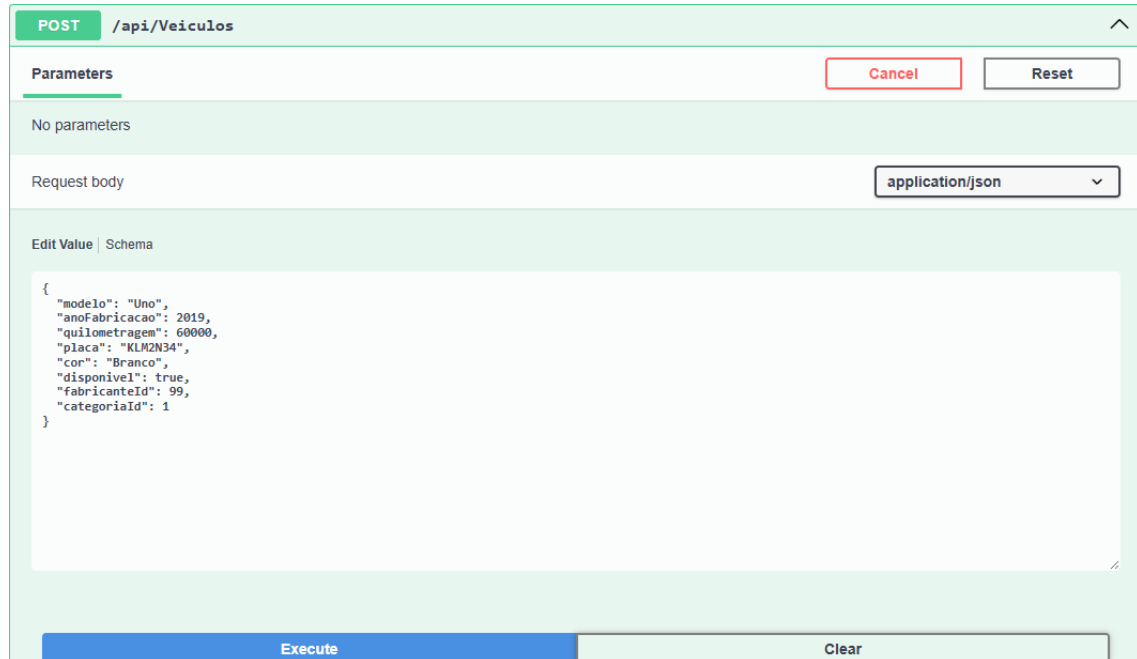
Figura 37 - Retorno do teste 18

Teste 19 - Erro: fabricante que não existe

Chamada: POST /api/Veiculos

Resultado esperado: Código 400, porque não existe fabricante com id 99.

Resultado obtido: Código 400, conforme o esperado.

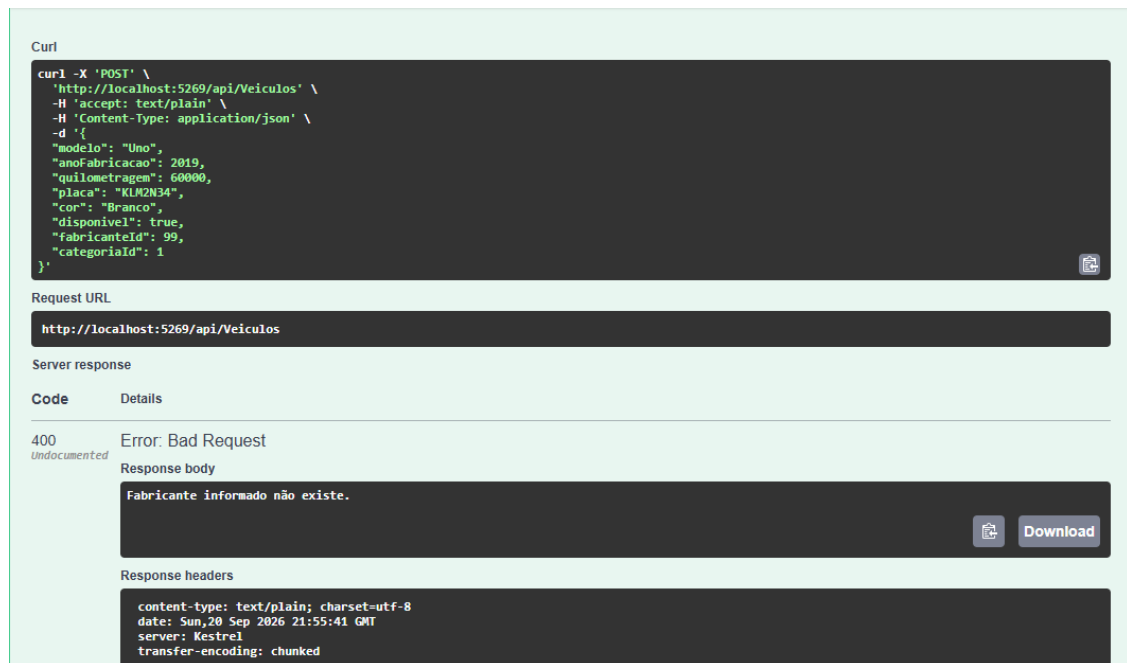


The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** /api/Veiculos
- Parameters:** No parameters
- Request body:** application/json
- Request body content:**

```
{  "modelo": "Uno",  "anoFabricacao": 2019,  "quilometragem": 60000,  "placa": "KLM2N34",  "cor": "Branco",  "disponivel": true,  "fabricanteId": 99,  "categoriaId": 1}
```
- Buttons:** Execute, Clear, Cancel, Reset

Figura 38 - Chamada do teste 19



The screenshot shows the response of the POST request in a REST client interface:

- Request URL:** http://localhost:5269/api/Veiculos
- Server response:**
 - Code:** 400
 - Details:** Error: Bad Request
 - Response body:** Fabricante informado não existe.
 - Response headers:**

```
content-type: text/plain; charset=utf-8
date: Sun, 20 Sep 2026 21:55:41 GMT
server: Kestrel
transfer-encoding: chunked
```

Figura 39 - Retorno do teste 19

Teste 20 - Erro: ano e quilometragem inválidos

Chamada: POST /api/Veiculos

Resultado esperado: Código 400 com os erros de validação do ano de fabricação e da quilometragem.

Resultado obtido: Código 400, conforme o esperado.

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** /api/Veiculos
- Parameters:** No parameters
- Request body:** application/json
- Request body content:**

```
{  "modelo": "Uno",  "anoFabricacao": 1800,  "quilometragem": -10,  "placa": "KLM2N34",  "cor": "Branco",  "disponivel": true,  "fabricanteId": 1,  "categoriaId": 1}
```
- Buttons:** Execute, Clear, Cancel, Reset

Figura 40 - Chamada do teste 20

The screenshot shows the response of the POST request in a REST client interface:

- Request URL:** http://localhost:5269/api/Veiculos
- Server response:**
 - Code:** 400
 - Details:** Error: Bad Request
 - Response body:**

```
{  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",  "title": "One or more validation errors occurred.",  "status": 400,  "errors": {    "AnoFabricacao": [      "The field AnoFabricacao must be between 1950 and 2100."    ],    "Quilometragem": [      "The field Quilometragem must be between 0 and 2147483647."    ]  },  "traceId": "00-33d8ab2d29021588dc6abc44f04d239c-6c99ca048cb90a4c-00"}
```
 - Response headers:**

```
content-type: application/problem+json; charset=utf-8date: Sun, 20 Sep 2020 21:55:44 GMTserver: Westreltransfer-encoding: chunked
```

Figura 41 - Retorno do teste 20

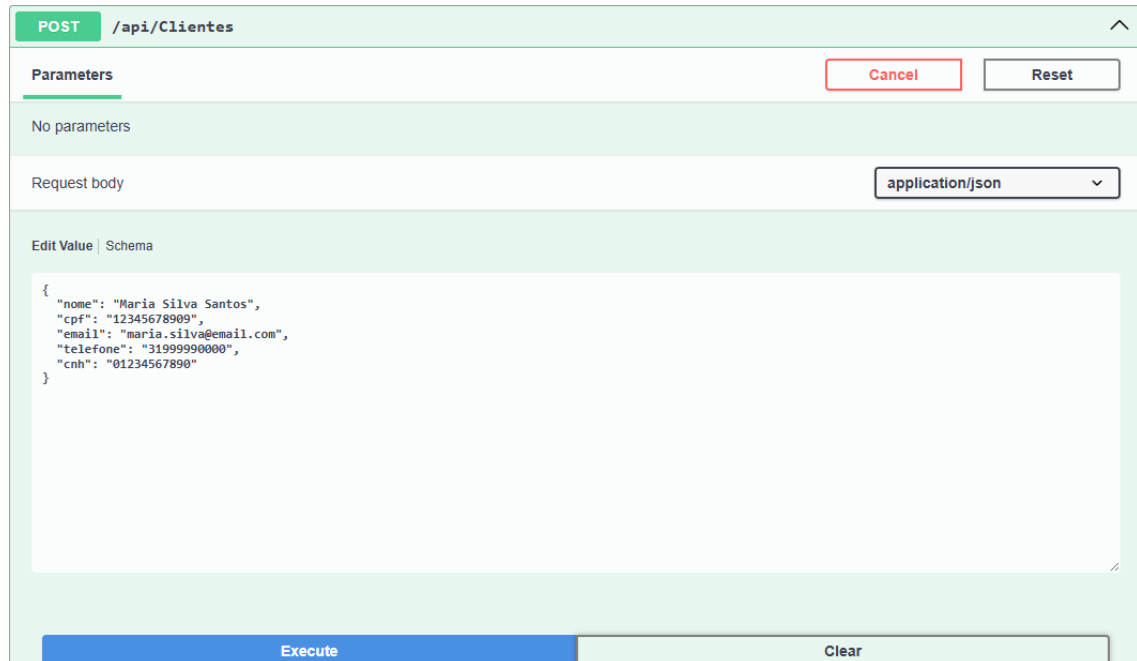
5. Clientes

Teste 21 - Cadastrar cliente

Chamada: POST /api/Clientes

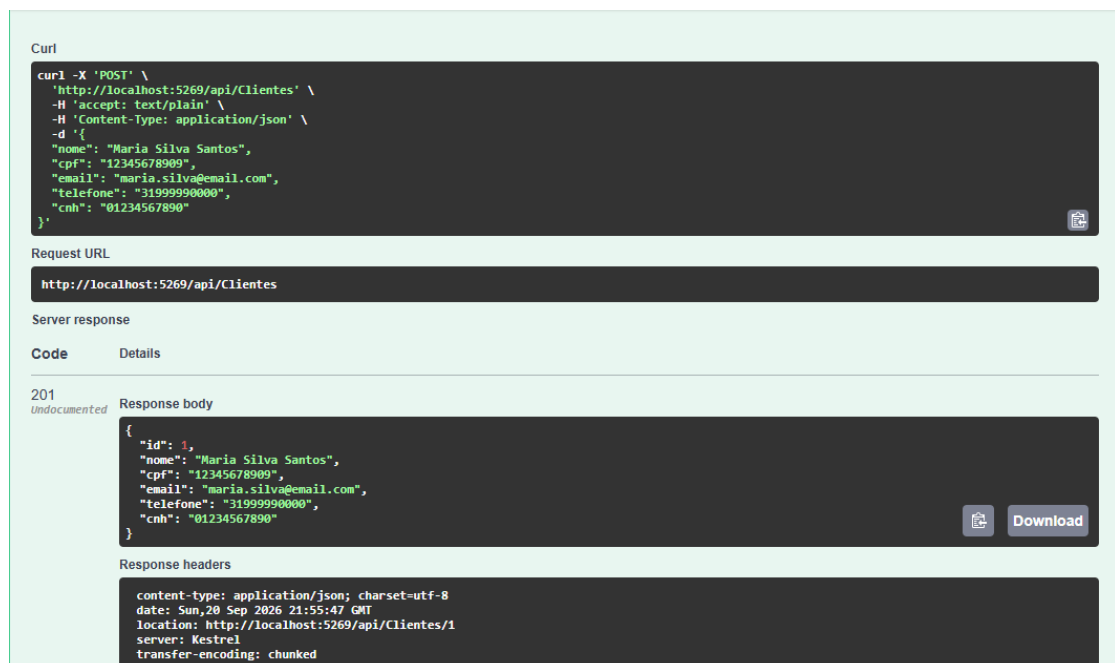
Resultado esperado: Código 201 com o cliente criado.

Resultado obtido: Código 201, conforme o esperado.



The screenshot shows a REST client interface with a green header bar. The method is POST and the URL is /api/Clientes. Below the header, there are tabs for Parameters, Request body, and Schema. The Parameters tab is active, showing 'No parameters'. The Request body tab is also visible, showing a JSON body: { "nome": "Maria Silva Santos", "cpf": "12345678909", "email": "maria.silva@email.com", "telefone": "31999990000", "cnh": "01234567890" }. The Schema tab is also visible. At the bottom, there are buttons for 'Execute' and 'Clear'.

Figura 42 - Chamada do teste 21



The screenshot shows the response of the POST request. The status code is 201. The response body is a JSON object: { "id": 1, "nome": "Maria Silva Santos", "cpf": "12345678909", "email": "maria.silva@email.com", "telefone": "31999990000", "cnh": "01234567890" }. The response headers are: content-type: application/json; charset=utf-8, date: Sun, 20 Sep 2026 21:55:47 GMT, location: http://localhost:5269/api/Clientes/1, server: Kestrel, transfer-encoding: chunked. There is a 'Download' button next to the response body.

Figura 43 - Retorno do teste 21

Teste 22 - Listar clientes

Chamada: GET /api/Clientes

Resultado esperado: Código 200 com a lista de clientes.

Resultado obtido: Código 200, conforme o esperado.

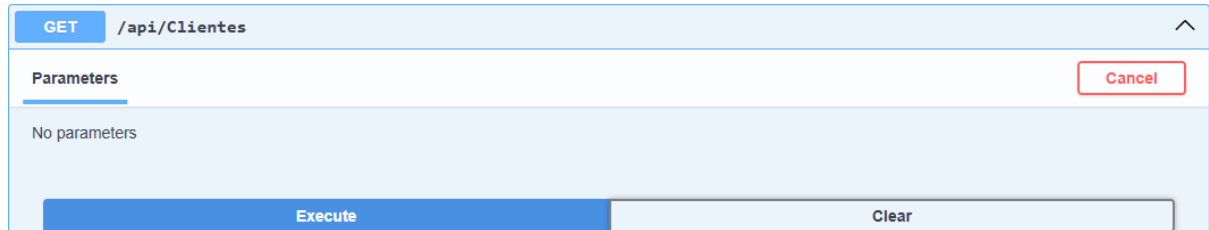


Figura 44 - Chamada do teste 22

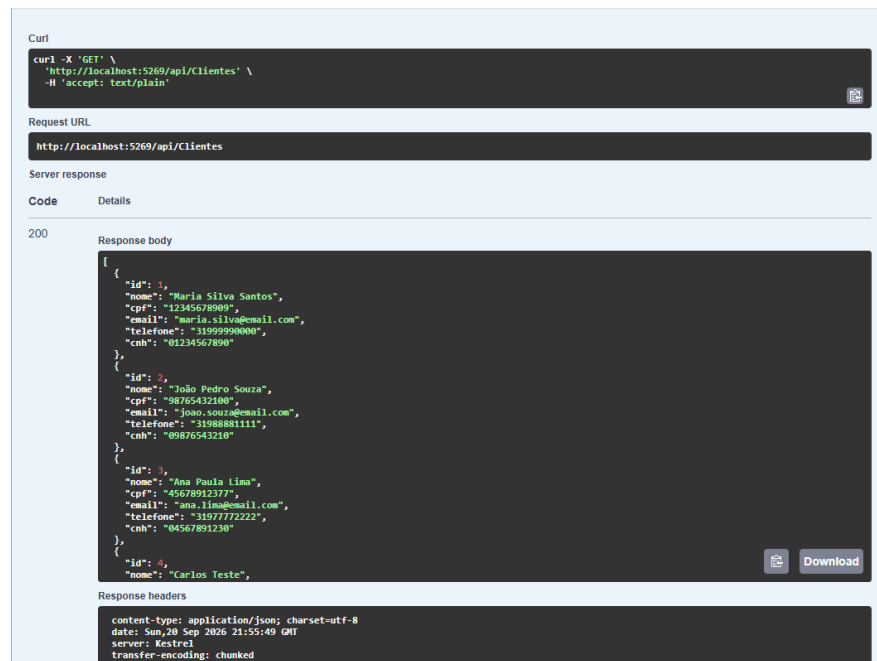


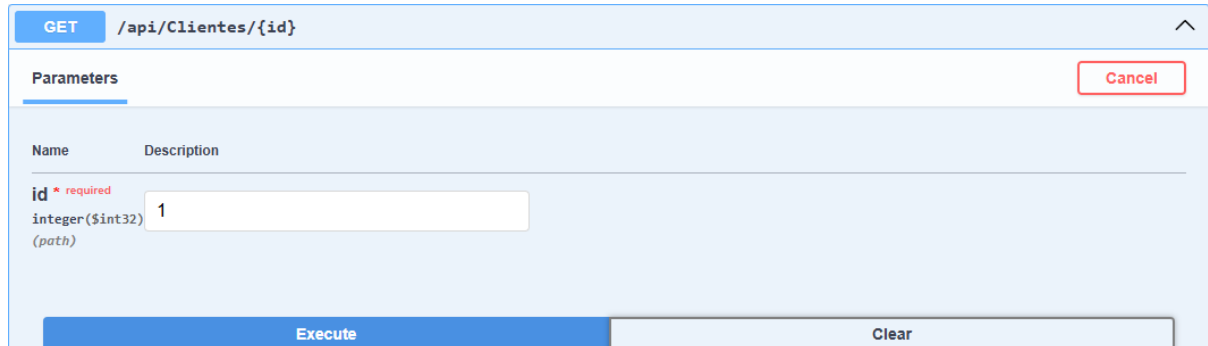
Figura 45 - Retorno do teste 22

Teste 23 - Buscar cliente por id

Chamada: GET /api/Clientes/{id} (id = 1)

Resultado esperado: Código 200 com os dados do cliente de id 1.

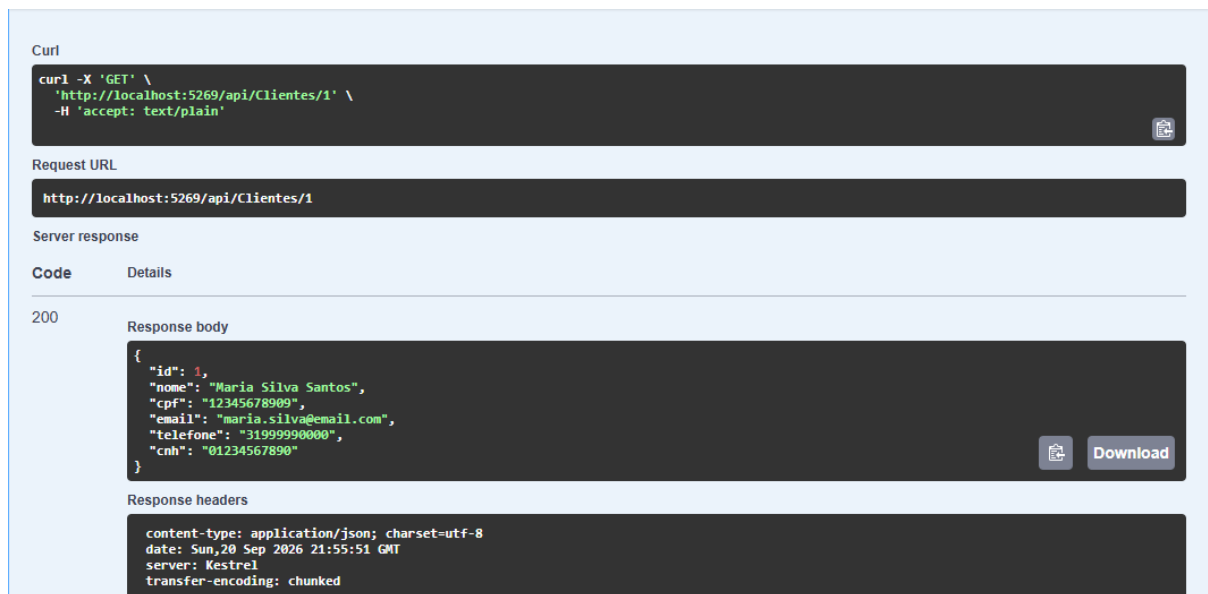
Resultado obtido: Código 200, conforme o esperado.



The screenshot shows a REST client interface. At the top, there's a tab labeled 'GET' and the URL '/api/Clientes/{id}'. Below this is a 'Parameters' section with a 'Cancel' button. A table with 'Name' and 'Description' headers contains one parameter: 'id' with a description 'integer(\$int32) (path)'. The value '1' is entered in the input field next to 'id'. At the bottom, there are 'Execute' and 'Clear' buttons.

Name	Description
id * required	integer(\$int32) (path)

Figura 46 - Chamada do teste 23



The screenshot shows the response of the GET request. It includes a 'Curl' section with the command: `curl -X 'GET' \ 'http://localhost:5269/api/Clientes/1' \ -H 'accept: text/plain'`. The 'Request URL' is `http://localhost:5269/api/Clientes/1`. The 'Server response' section shows a 'Code' of 200 and a 'Response body' containing a JSON object: `{ "id": 1, "nome": "Maria Silva Santos", "cpf": "12345678909", "email": "maria.silva@email.com", "telefone": "31999990000", "cnh": "01234567890" }`. There is a 'Download' button next to the response body. The 'Response headers' section shows: `content-type: application/json; charset=utf-8`, `date: Sun, 20 Sep 2026 21:55:51 GMT`, `server: Kestrel`, and `transfer-encoding: chunked`.

```
curl -X 'GET' \
'http://localhost:5269/api/Clientes/1' \
-H 'accept: text/plain'
```

Request URL

`http://localhost:5269/api/Clientes/1`

Server response

Code: 200

Response body

```
{
  "id": 1,
  "nome": "Maria Silva Santos",
  "cpf": "12345678909",
  "email": "maria.silva@email.com",
  "telefone": "31999990000",
  "cnh": "01234567890"
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Sun, 20 Sep 2026 21:55:51 GMT
server: Kestrel
transfer-encoding: chunked
```

Figura 47 - Retorno do teste 23

Teste 24 - Atualizar cliente

Chamada: PUT /api/Clientes/{id} (id = 2)

Resultado esperado: Código 204, cliente atualizado (telefone).

Resultado obtido: Código 204, conforme o esperado.

The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** /api/Clientes/{id}
- Parameters:** A table with two columns: Name and Description. The 'id' parameter is listed as 'required', 'integer(\$int32)', and '(path)', with a value of '2' entered in the input field.
- Request body:** A dropdown menu is set to 'application/json'.
- Edit Value | Schema:** A text area containing a JSON object:

```
{  "nome": "João Pedro Souza",  "cpf": "98765432100",  "email": "joao.souza@email.com",  "telefone": "3195554444",  "cnh": "09876543210"}
```
- Buttons:** 'Execute' (blue) and 'Clear' (grey) buttons are at the bottom.

Figura 48 - Chamada do teste 24

The screenshot shows a terminal window with the following content:

```
Curl
curl -X 'PUT' \
  'http://localhost:5269/api/Clientes/2' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "nome": "João Pedro Souza",
    "cpf": "98765432100",
    "email": "joao.souza@email.com",
    "telefone": "3195554444",
    "cnh": "09876543210"
  }'

Request URL
http://localhost:5269/api/Clientes/2

Server response

Code    Details
204     Undocumented
Response headers
date: Sun,20 Sep 2026 21:55:54 GMT
server: Kestrel
```

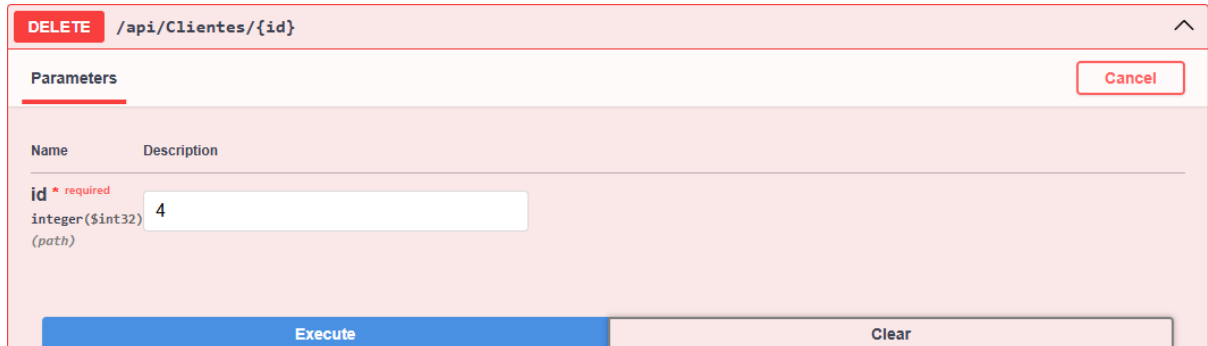
Figura 49 - Retorno do teste 24

Teste 25 - Excluir cliente

Chamada: DELETE /api/Clientes/{id} (id = 4)

Resultado esperado: Código 204, cliente excluído.

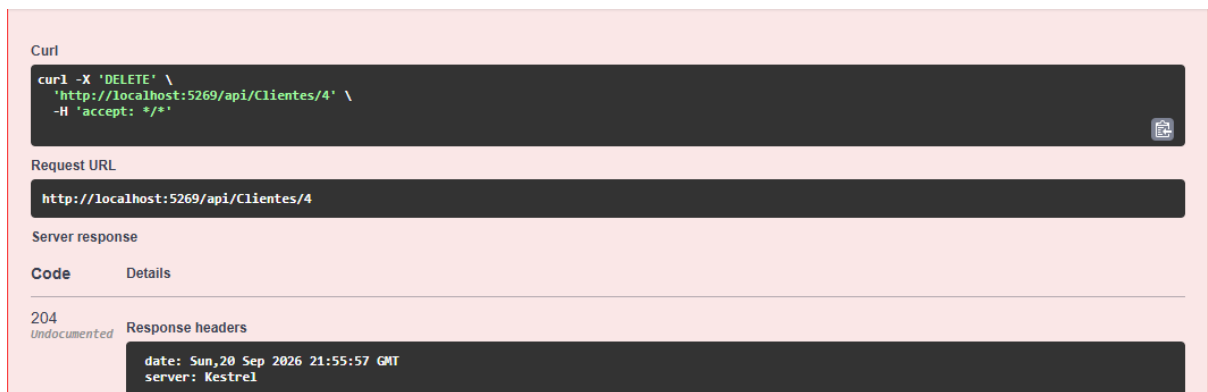
Resultado obtido: Código 204, conforme o esperado.



The screenshot shows a REST client interface with a red header bar. The method is 'DELETE' and the URL is '/api/Clientes/{id}'. Below the header, there is a 'Parameters' section with a table. The table has two columns: 'Name' and 'Description'. The first row has 'id' as the name, marked as 'required', and 'integer(\$int32)' as the description. The value '4' is entered in the input field. At the bottom, there are two buttons: 'Execute' (blue) and 'Clear' (light gray). A 'Cancel' button is also visible in the top right corner.

Name	Description
id * required	integer(\$int32)

Figura 50 - Chamada do teste 25



The screenshot shows the response of the DELETE request. The 'Curl' section displays the command: `curl -X 'DELETE' \ 'http://localhost:5269/api/Clientes/4' \ -H 'accept: */*'`. The 'Request URL' section shows 'http://localhost:5269/api/Clientes/4'. The 'Server response' section shows a 'Code' of '204' and 'Details' of 'Undocumented'. The 'Response headers' section shows 'date: Sun, 20 Sep 2026 21:55:57 GMT' and 'server: Kestrel'.

```
curl -X 'DELETE' \
'http://localhost:5269/api/Clientes/4' \
-H 'accept: */*'

Request URL
http://localhost:5269/api/Clientes/4

Server response
Code    Details
204     Undocumented
Response headers
date: Sun, 20 Sep 2026 21:55:57 GMT
server: Kestrel
```

Figura 51 - Retorno do teste 25

Teste 26 - Erro: CPF já cadastrado

Chamada: POST /api/Clientes

Resultado esperado: Código 409, porque o CPF 12345678909 já está cadastrado.

Resultado obtido: Código 409, conforme o esperado.

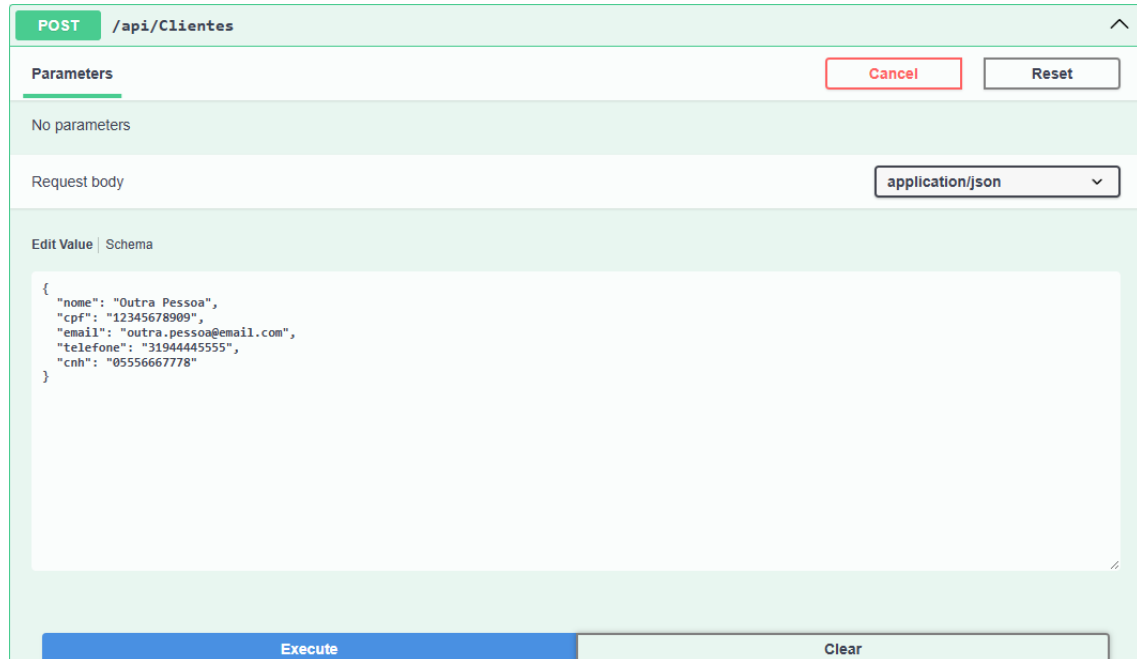


Figura 52 - Chamada do teste 26

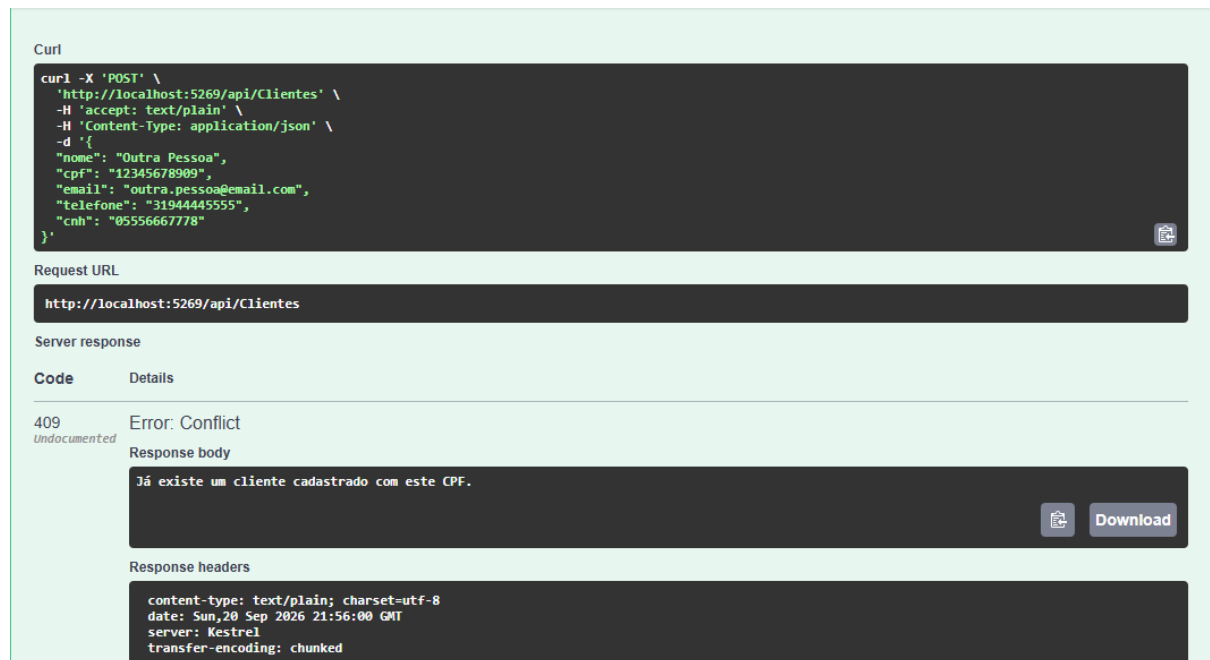


Figura 53 - Retorno do teste 26

Teste 27 - Erro: CPF e e-mail inválidos

Chamada: POST /api/Clientes

Resultado esperado: Código 400 com os erros de validação do CPF e do e-mail.

Resultado obtido: Código 400, conforme o esperado.

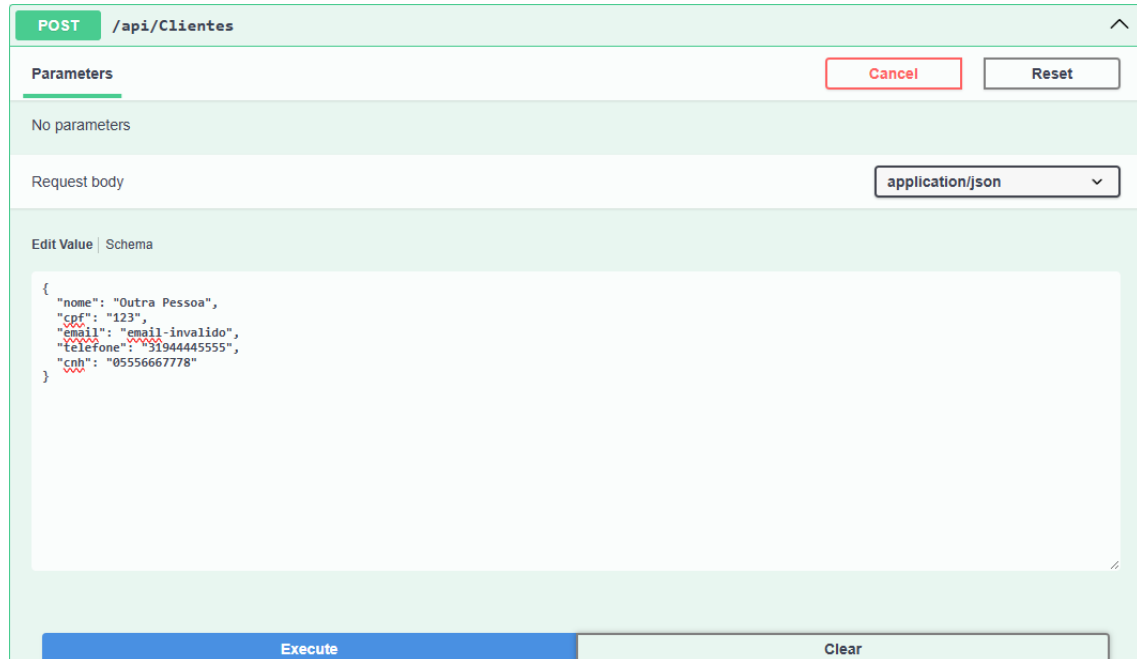


Figura 54 - Chamada do teste 27

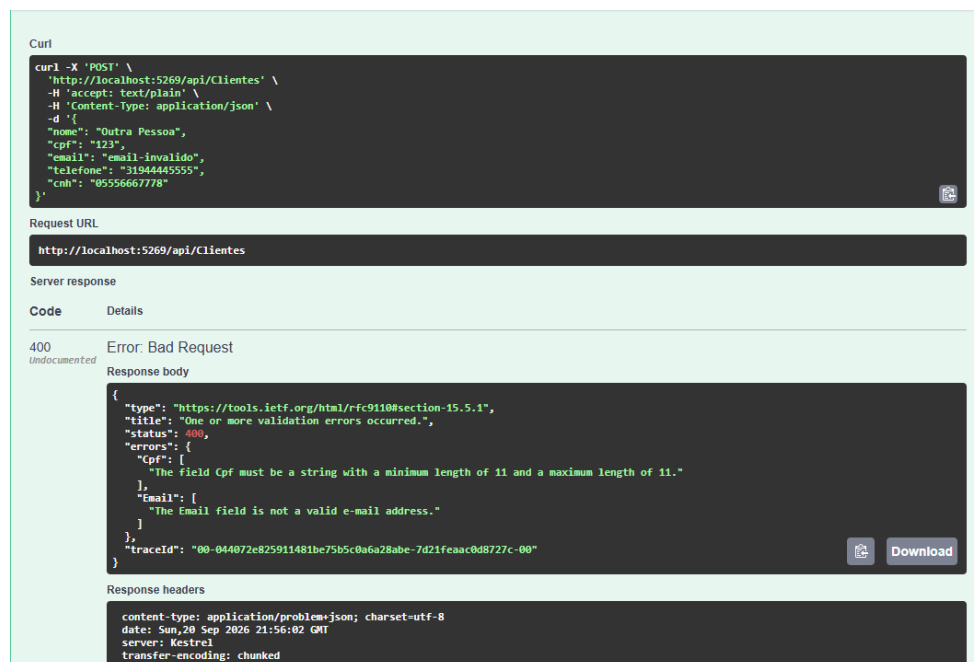


Figura 55 - Retorno do teste 27

6. Aluguéis

Teste 28 - Cadastrar aluguel

Chamada: POST /api/Alugueis

Resultado esperado: Código 201. O sistema preenche a quilometragem inicial (15000), a diária da categoria (130) e o valor total (5 diárias = 650).

Resultado obtido: Código 201, conforme o esperado.

The screenshot shows a REST client interface with a green header bar. The method is 'POST' and the URL is '/api/Alugueis'. Below the header, there are 'Parameters' and 'Request body' tabs. The 'Request body' tab is active, showing a JSON payload:

```
{  "clienteId": 1,  "veiculoId": 1,  "dataInicio": "2026-09-15T10:00:00",  "dataFim": "2026-09-20T10:00:00"}
```

. At the bottom, there is a blue 'Execute' button and a 'Clear' button.

Figura 56 - Chamada do teste 28

The screenshot shows the same REST client interface after the request. The 'Server response' tab is active, showing a status code of '201' and a 'Response body' containing a JSON object:

```
{  "id": 1,  "clienteId": 1,  "veiculoId": 1,  "dataInicio": "2026-09-15T10:00:00",  "dataDevolucao": null,  "quilometragemInicial": 15000,  "quilometragemFinal": null,  "valorDiaria": 130,  "valorTotal": 650}
```

. Below the response body, there are 'Response headers' showing:

```
content-type: application/json; charset=utf-8  date: Sun, 20 Sep 2026 21:56:05 GMT  location: http://localhost:5269/api/Alugueis/1  server: Kestrel  transfer-encoding: chunked
```

. A 'Download' button is visible next to the response body.

Figura 57 - Retorno do teste 28

Teste 29 - Erro: veículo já alugado

Chamada: POST /api/Alugueis

Resultado esperado: Código 400, porque o veículo 1 já está alugado.

Resultado obtido: Código 400, conforme o esperado.

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** /api/Alugueis
- Parameters:** No parameters
- Request body:** application/json
- Request body content:**

```
{  "clienteId": 3,  "veiculoId": 1,  "dataInicio": "2026-09-16T10:00:00",  "dataFim": "2026-09-18T10:00:00"}
```
- Buttons:** Execute, Clear, Cancel, Reset

Figura 58 - Chamada do teste 29

The screenshot shows the response of the POST request with the following details:

- Request URL:** http://localhost:5269/api/Alugueis
- Server response:**
 - Code:** 400
 - Details:** Error: Bad Request
 - Response body:** Veículo indisponível para aluguel.
 - Response headers:**

```
content-type: text/plain; charset=utf-8
date: Sun, 20 Sep 2026 21:56:07 GMT
server: Kestrel
transfer-encoding: chunked
```

Figura 59 - Retorno do teste 29

Teste 30 - Erro: data de fim antes do início

Chamada: POST /api/Alugueis

Resultado esperado: Código 400, porque a data de fim é anterior à data de início.

Resultado obtido: Código 400, conforme o esperado.

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** /api/Alugueis
- Parameters:** No parameters
- Request body:** application/json
- Request body content:**

```
{  "clienteId": 3,  "veiculoId": 3,  "dataInicio": "2026-09-20T10:00:00",  "dataFim": "2026-09-18T10:00:00"}
```
- Buttons:** Execute, Clear, Cancel, Reset

Figura 60 - Chamada do teste 30

The screenshot shows the response of the POST request, which is a 400 Bad Request error. The response details are as follows:

- Code:** 400
- Details:** Error: Bad Request
- Response body:**

```
A data de fim deve ser maior que a data de inicio.
```
- Response headers:**

```
content-type: text/plain; charset=utf-8
date: Sun, 20 Sep 2026 21:56:10 GMT
server: Kestrel
transfer-encoding: chunked
```

Figura 61 - Retorno do teste 30

Teste 31 - Listar alugueis

Chamada: GET /api/Alugueis

Resultado esperado: Código 200 com a lista de alugueis.

Resultado obtido: Código 200, conforme o esperado.

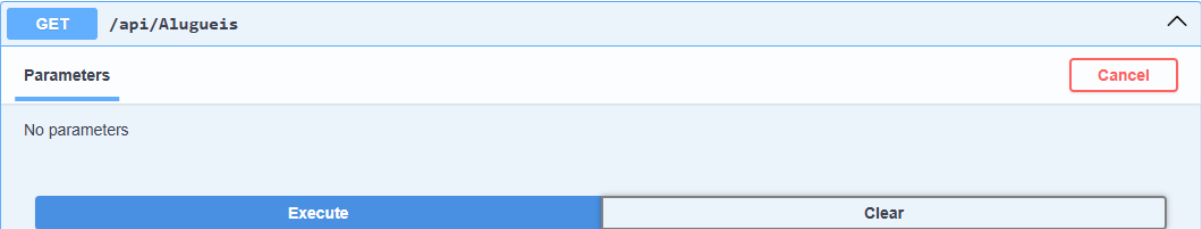


Figura 62 - Chamada do teste 31

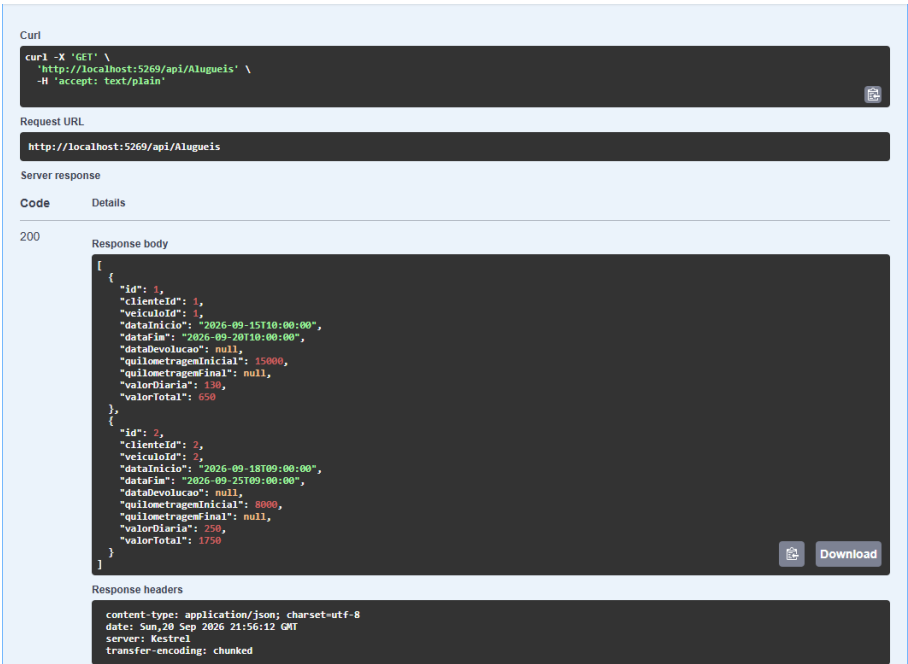


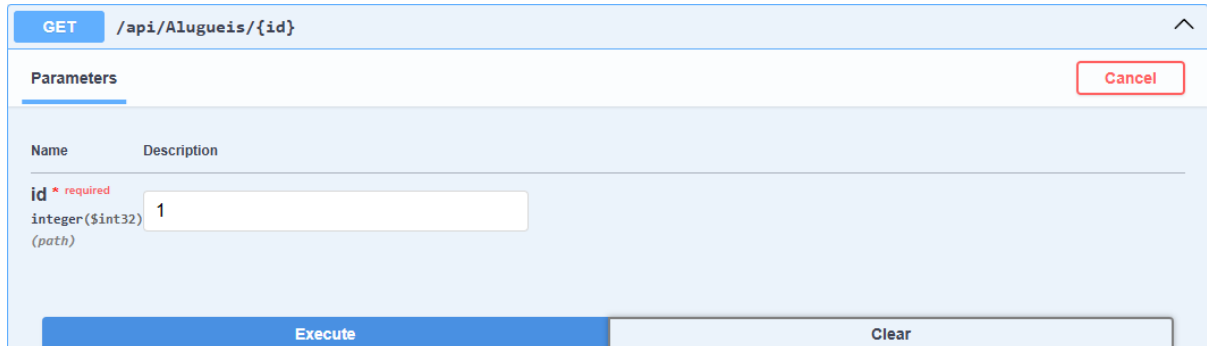
Figura 63 - Retorno do teste 31

Teste 32 - Buscar aluguel por id

Chamada: GET /api/Alugueis/{id} (id = 1)

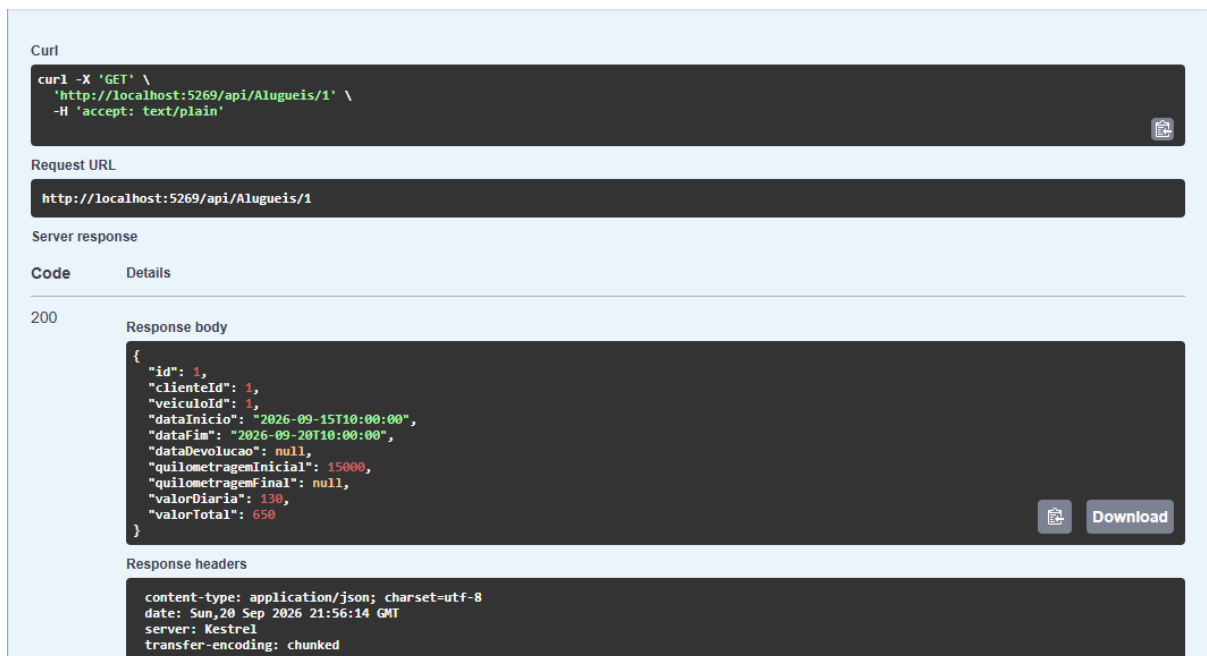
Resultado esperado: Código 200 com os dados do aluguel de id 1.

Resultado obtido: Código 200, conforme o esperado.



The screenshot shows a REST client interface with a GET request to `/api/Alugueis/{id}`. The `Parameters` tab is active, showing a table with two columns: `Name` and `Description`. A parameter `id` is defined as `integer($int32)` and `(path)`, with a value of `1` entered in the input field. The `Execute` button is highlighted in blue, and a `Clear` button is visible to its right.

Figura 64 - Chamada do teste 32



The screenshot shows the response of the GET request to `http://localhost:5269/api/Alugueis/1`. The `Server response` section shows a `200` status code. The `Response body` is a JSON object with the following fields: `id`, `clienteId`, `veiculoId`, `dataInicio`, `dataFim`, `dataDevolucao`, `quilometragemInicial`, `quilometragemFinal`, `valorDiaria`, and `valorTotal`. The `Response headers` section shows the following headers: `content-type: application/json; charset=utf-8`, `date: Sun, 20 Sep 2026 21:56:14 GMT`, `server: Kestrel`, and `transfer-encoding: chunked`.

Figura 65 - Retorno do teste 32

Teste 33 - Atualizar aluguel (nova data de fim)

Chamada: PUT /api/Alugueis/{id} (id = 1)

Resultado esperado: Código 204. Com a nova data de fim o valor total é recalculado (7 diárias = 910).

Resultado obtido: Código 204, conforme o esperado.

The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** /api/Alugueis/{id}
- Parameters:** A table with two columns: Name and Description. The first row has 'id' in the Name column, marked as 'required' in red, and '1' in the Description column. Below the input field, it says 'integer(\$int32)' and '(path)'.
- Request body:** A dropdown menu set to 'application/json'.
- Edit Value | Schema:** A text area containing a JSON object:

```
{  "dataInicio": "2026-09-15T10:00:00",  "dataFim": "2026-09-22T10:00:00"}
```
- Buttons:** 'Execute' (blue) and 'Clear' (grey) at the bottom.

Figura 66 - Chamada do teste 33

The screenshot shows the result of the PUT request in a REST client interface:

- Curl:** A dark box containing the curl command:

```
curl -X 'PUT' \  'http://localhost:5269/api/Alugueis/1' \  -H 'accept: */*' \  -H 'Content-Type: application/json' \  -d '{  "dataInicio": "2026-09-15T10:00:00",  "dataFim": "2026-09-22T10:00:00"  }'
```
- Request URL:** A dark box containing the URL: `http://localhost:5269/api/Alugueis/1`
- Server response:** A section with a 'Code' column showing '204' and a 'Details' column showing 'Undocumented'.
- Response headers:** A dark box containing the headers:

```
date: Sun,20 Sep 2026 21:56:16 GMT  server: Kestrel
```

Figura 67 - Retorno do teste 33

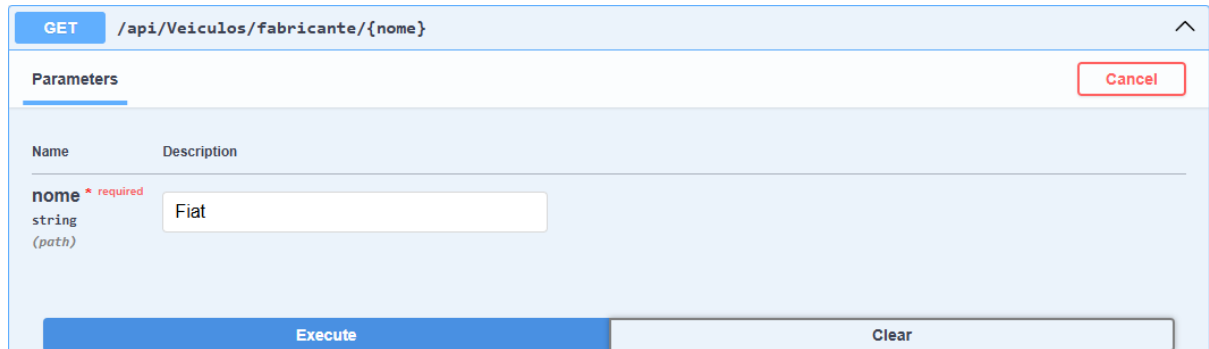
7. Filtros com join

Teste 34 - Filtro 1: veículos por fabricante (inner join)

Chamada: GET /api/Veiculos/fabricante/{nome} (nome = Fiat)

Resultado esperado: Código 200 com os veículos da Fiat (inner join entre Veiculos e Fabricantes).

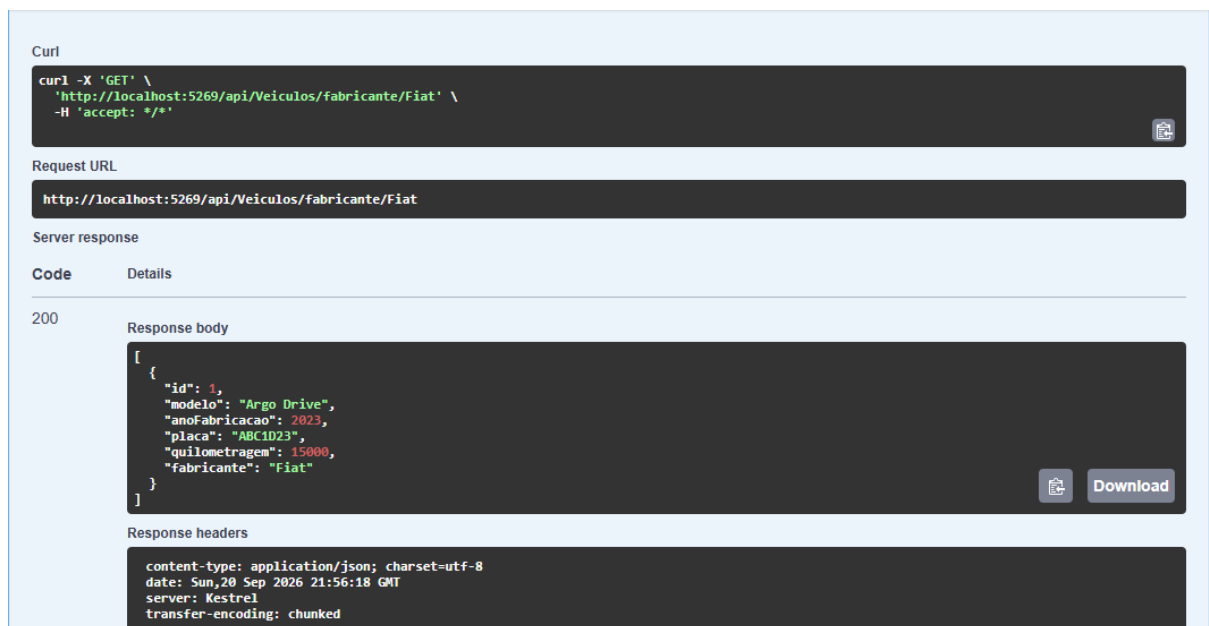
Resultado obtido: Código 200, conforme o esperado.



The screenshot shows a REST client interface. At the top, the method is 'GET' and the URL is '/api/Veiculos/fabricante/{nome}'. Below this, there is a 'Parameters' section with a table. The table has two columns: 'Name' and 'Description'. There is one parameter named 'nome' which is marked as 'required' and has a type of 'string (path)'. The value for 'nome' is 'Fiat'. At the bottom, there are two buttons: 'Execute' and 'Clear'.

Name	Description
nome * required string (path)	Fiat

Figura 68 - Chamada do teste 34



The screenshot shows the response of the GET request. The status code is 200. The response body is a JSON array containing one object. The object has the following properties: 'id' (1), 'modelo' ('Argo Drive'), 'anoFabricacao' (2023), 'placa' ('ABC1D23'), 'quilometragem' (15000), and 'fabricante' ('Fiat'). The response headers are also shown.

```
curl -X 'GET' \
  'http://localhost:5269/api/Veiculos/fabricante/Fiat' \
  -H 'accept: */*'

Request URL
http://localhost:5269/api/Veiculos/fabricante/Fiat

Server response
Code 200
Details

Response body
[
  {
    "id": 1,
    "modelo": "Argo Drive",
    "anoFabricacao": 2023,
    "placa": "ABC1D23",
    "quilometragem": 15000,
    "fabricante": "Fiat"
  }
]

Response headers
content-type: application/json; charset=utf-8
date: Sun, 20 Sep 2026 21:56:18 GMT
server: Kestrel
transfer-encoding: chunked
```

Figura 69 - Retorno do teste 34

Teste 35 - Filtro 2: veículos disponíveis (inner join)

Chamada: GET /api/Veiculos/disponiveis

Resultado esperado: Código 200 só com os veículos que não estão alugados (inner join entre Veiculos, Categorias e Fabricantes).

Resultado obtido: Código 200, conforme o esperado.

The screenshot shows a REST client interface with a blue header bar. The method is 'GET' and the URL is '/api/Veiculos/disponiveis'. Below the header, there is a 'Parameters' section with a 'Cancel' button. A table lists the parameters:

Name	Description
categoria	
string	
(query)	

At the bottom, there are 'Execute' and 'Clear' buttons.

Figura 70 - Chamada do teste 35

The screenshot shows the results of the REST client call. It includes the curl command, the request URL, and the server response.

Curl

```
curl -X 'GET' \
'http://localhost:5269/api/Veiculos/disponiveis' \
-H 'accept: */*'
```

Request URL

```
http://localhost:5269/api/Veiculos/disponiveis
```

Server response

Code	Details
200	Response body <pre>[{ "id": 3, "modelo": "Corolla", "placa": "RIV7U89", "fabricante": "Toyota", "categoria": "Sedan", "valorDiaria": 180 }]</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Sun, 20 Sep 2026 21:56:20 GMT server: Kestrel transfer-encoding: chunked</pre>

Figura 71 - Retorno do teste 35

Teste 36 - Filtro 3: alugueis de um cliente pelo CPF (inner join)

Chamada: GET /api/Alugueis/cliente/{cpf} (cpf = 12345678909)

Resultado esperado: Código 200 com os alugueis da cliente Maria (inner join entre Alugueis, Clientes e Veiculos).

Resultado obtido: Código 200, conforme o esperado.

The screenshot shows a REST client interface with a GET request to the endpoint `/api/Alugueis/cliente/{cpf}`. The `Parameters` tab is active, displaying a table with two columns: `Name` and `Description`. A single parameter is listed: `cpf` (string, path), marked as required, with the value `12345678909` entered in the input field. At the bottom, there are `Execute` and `Clear` buttons.

Name	Description
cpf * required string (path)	12345678909

Figura 72 - Chamada do teste 36

The screenshot shows the response of the GET request. The `Response body` tab is active, displaying a JSON array with one object. The `Response headers` tab is also visible, showing the headers of the response.

Response body:

```
[
  {
    "id": 1,
    "cliente": "Maria Silva Santos",
    "cpf": "12345678909",
    "veiculo": "Argo Drive",
    "placa": "ABC1023",
    "dataInicio": "2026-09-15T10:00:00",
    "dataFim": "2026-09-22T10:00:00",
    "dataDevolucao": null,
    "valorTotal": 910
  }
]
```

Response headers:

```
content-type: application/json; charset=utf-8
date: Sun, 20 Sep 2026 21:56:22 GMT
server: Kestrel
transfer-encoding: chunked
```

Figura 73 - Retorno do teste 36

Teste 37 - Filtro 4: alugueis em aberto (inner join)

Chamada: GET /api/Alugueis/abertos

Resultado esperado: Código 200 com os alugueis que ainda não foram devolvidos (inner join entre Alugueis, Clientes e Veiculos).

Resultado obtido: Código 200, conforme o esperado.

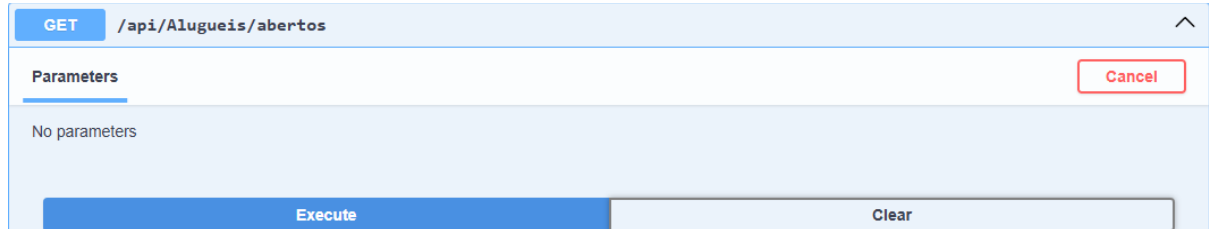


Figura 74 - Chamada do teste 37

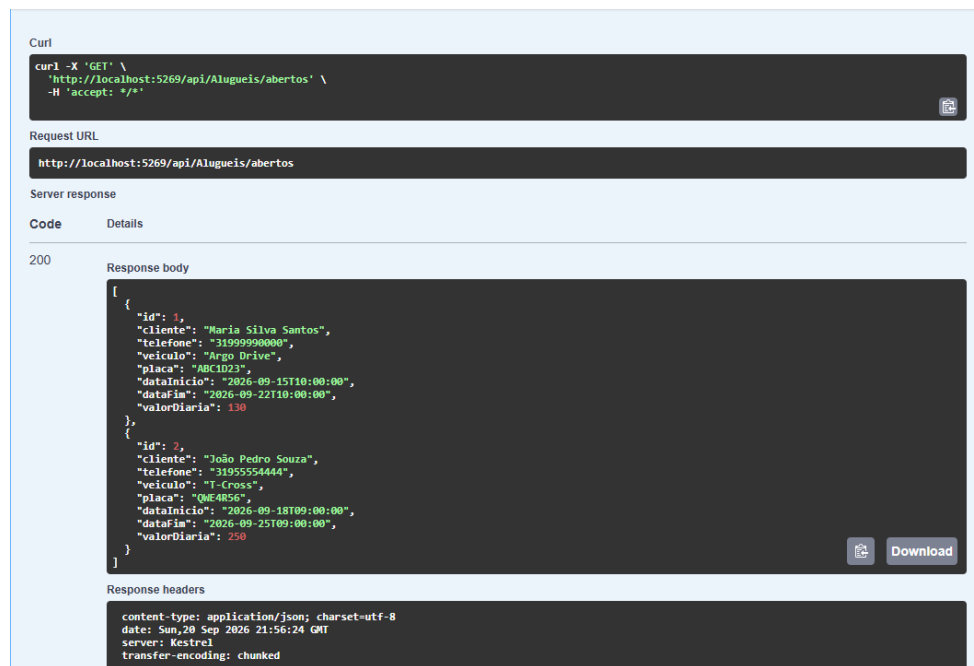


Figura 75 - Retorno do teste 37

Teste 38 - Filtro 5: clientes sem aluguel (left join)

Chamada: GET /api/Clientes/sem-aluguel

Resultado esperado: Código 200 com a cliente Ana, que é a única sem aluguel (left join entre Clientes e Alugueis).

Resultado obtido: Código 200, conforme o esperado.

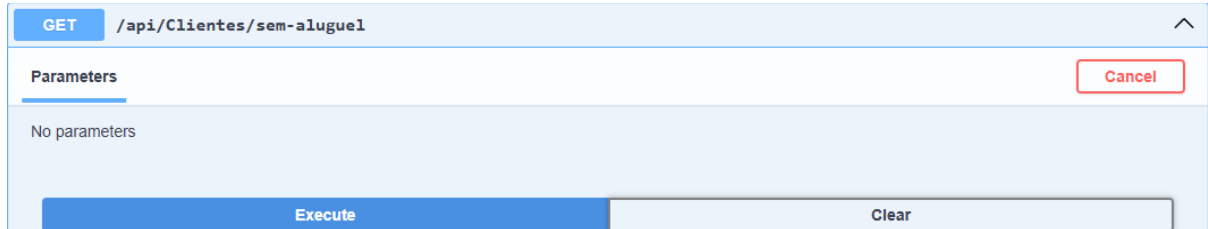


Figura 76 - Chamada do teste 38

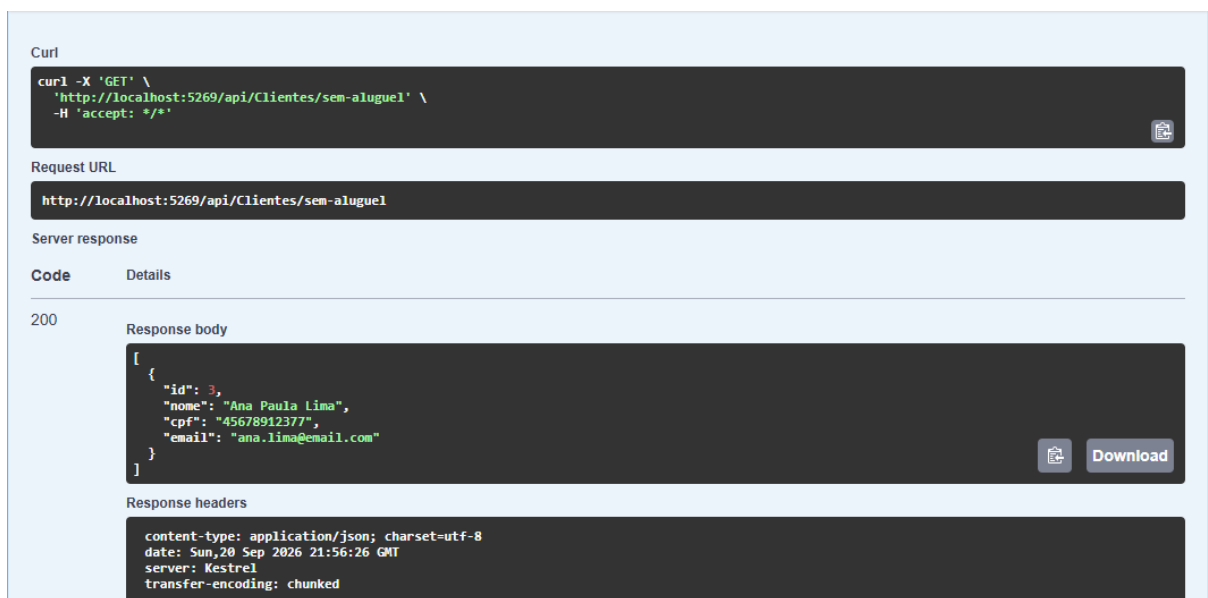


Figura 77 - Retorno do teste 38

8. Devolução e exclusão de aluguel

Teste 39 - Erro: quilometragem final menor que a inicial

Chamada: PUT /api/Alugueis/{id}/devolucao (id = 1, quilometragemFinal = 100)

Resultado esperado: Código 400, porque a quilometragem final (100) é menor que a inicial (15000).

Resultado obtido: Código 400, conforme o esperado.

The screenshot shows a REST client interface with a PUT request to the endpoint `/api/Alugueis/{id}/devolucao`. The parameters section is expanded, showing three fields: `id` (required, integer, path) with value 1, `quilometragemFinal` (integer, query) with value 100, and `dataDevolucao` (string, query) with value `dataDevolucao`. The interface includes an 'Execute' button and a 'Clear' button.

Figura 78 - Chamada do teste 39

The screenshot shows the response of the PUT request. The status code is 400 (Error: Bad Request). The response body is: `A quilometragem final não pode ser menor que a quilometragem inicial.`. The response headers are: `content-type: text/plain; charset=utf-8`, `date: Sun, 20 Sep 2026 21:56:28 GMT`, `server: Kestrel`, and `transfer-encoding: chunked`.

Figura 79 - Retorno do teste 39

Teste 40 - Registrar devolução do veículo

Chamada: PUT /api/Alugueis/{id}/devolucao (id = 1, quilometragemFinal = 15600, dataDevolucao = 2026-09-20T10:00:00)

Resultado esperado: Código 200 com a devolução registrada: quilometragem final 15600 e valor total recalculado (5 diárias = 650).

Resultado obtido: Código 200, conforme o esperado.

PUT /api/Alugueis/{id}/devolucao

Parameters

Name	Description
id * required	
integer(\$int32) (path)	1
quilometragemFinal	
integer(\$int32) (query)	15600
dataDevolucao	
string(\$date-time) (query)	2026-09-20T10:00:00

Execute Clear

Figura 80 - Chamada do teste 40

Curl

```
curl -X 'PUT' \
  'http://localhost:5269/api/Alugueis/1/devolucao?quilometragemFinal=15600&dataDevolucao=2026-09-20T10%3A00%3A00' \
  -H 'accept: */*'
```

Request URL

http://localhost:5269/api/Alugueis/1/devolucao?quilometragemFinal=15600&dataDevolucao=2026-09-20T10%3A00%3A00

Server response

Code Details

200

Response body

```
{
  "id": 1,
  "clienteId": 1,
  "veiculoId": 1,
  "dataInicio": "2026-09-15T10:00:00",
  "dataFim": "2026-09-22T10:00:00",
  "dataDevolucao": "2026-09-20T10:00:00",
  "quilometragemInicial": 15000,
  "quilometragemFinal": 15600,
  "valorDiaria": 130,
  "valorTotal": 650
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Sun, 20 Sep 2026 21:56:30 GMT
server: Kestrel
transfer-encoding: chunked
```

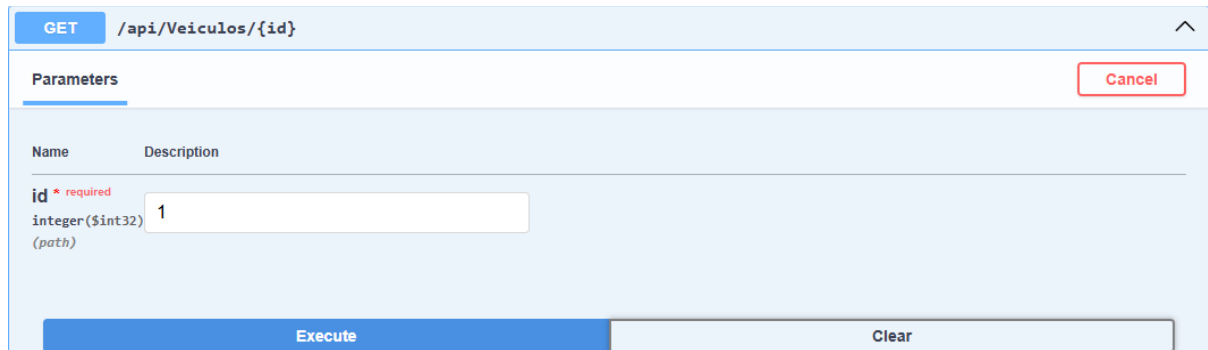
Figura 81 - Retorno do teste 40

Teste 41 - Veículo depois da devolução

Chamada: GET /api/Veiculos/{id} (id = 1)

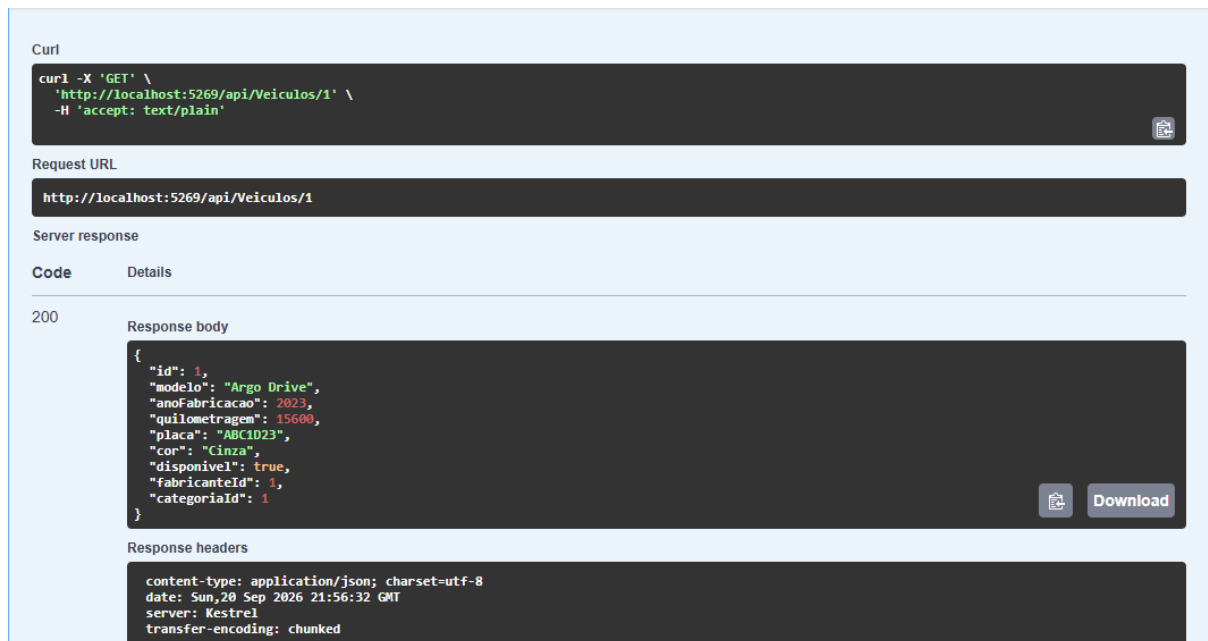
Resultado esperado: Código 200. O veículo voltou a ficar disponível e a quilometragem passou para 15600.

Resultado obtido: Código 200, conforme o esperado.



The screenshot shows a REST client interface with a GET request to `/api/Veiculos/{id}`. The `Parameters` tab is active, showing a table with two columns: `Name` and `Description`. A single parameter is listed: `id` (marked as required), with a type of `integer($int32)` and a value of `1`. The `(path)` label is visible below the parameter name. At the bottom, there are `Execute` and `Clear` buttons. A `Cancel` button is also present in the top right corner.

Figura 82 - Chamada do teste 41



The screenshot shows the response of the GET request. The `Request URL` is `http://localhost:5269/api/Veiculos/1`. The `Server response` section shows a `200` status code. The `Response body` is a JSON object:

```
{
  "id": 1,
  "modelo": "Argo Drive",
  "anoFabricacao": 2023,
  "quilometragem": 15600,
  "placa": "ABC1D23",
  "cor": "Cinza",
  "disponivel": true,
  "fabricanteId": 1,
  "categoriaId": 1
}
```

. The `Response headers` section shows:

```
content-type: application/json; charset=utf-8
date: Sun, 20 Sep 2026 21:56:32 GMT
server: Kestrel
transfer-encoding: chunked
```

. There are `Download` buttons for both the response body and headers.

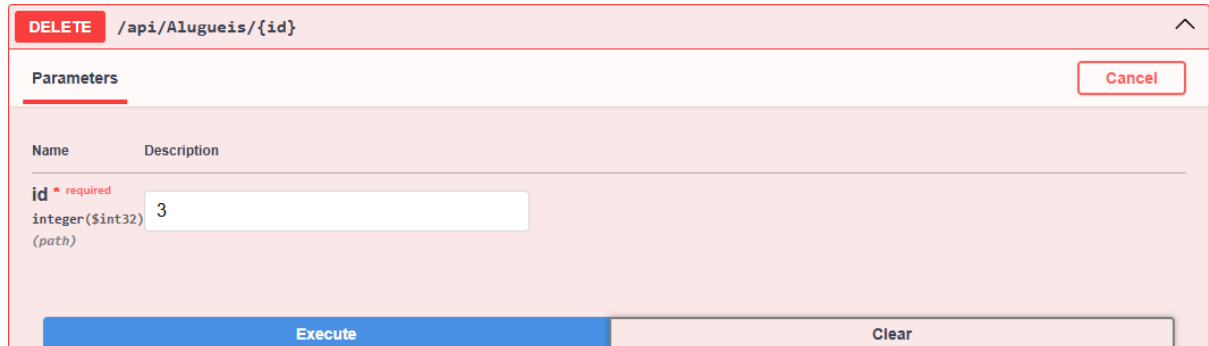
Figura 83 - Retorno do teste 41

Teste 42 - Excluir aluguel

Chamada: DELETE /api/Alugueis/{id} (id = 3)

Resultado esperado: Código 204, aluguel excluído.

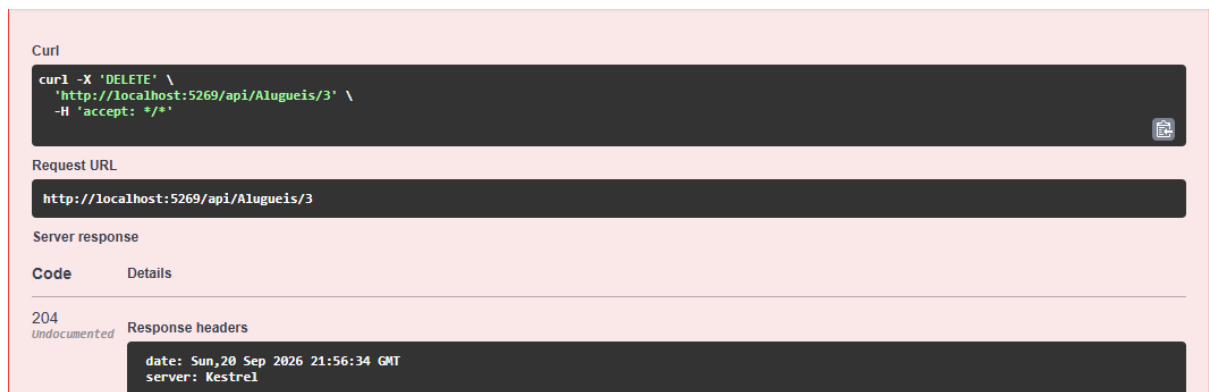
Resultado obtido: Código 204, conforme o esperado.



The screenshot shows a REST client interface with a red header bar. The method is 'DELETE' and the URL is '/api/Alugueis/{id}'. Below the header, there is a 'Parameters' tab. A table with two columns, 'Name' and 'Description', contains one parameter: 'id' with a description of 'integer(\$int32) (path)'. The value '3' is entered in the input field next to the parameter name. At the bottom, there are two buttons: 'Execute' (blue) and 'Clear' (light gray). A 'Cancel' button is located in the top right corner.

Name	Description
id * required	integer(\$int32) (path)

Figura 84 - Chamada do teste 42



The screenshot shows the response of the DELETE request. The 'Curl' section displays the command: `curl -X 'DELETE' \ 'http://localhost:5269/api/Alugueis/3' \ -H 'accept: */*'` . The 'Request URL' section shows 'http://localhost:5269/api/Alugueis/3'. The 'Server response' section shows a 'Code' of '204' and 'Details' of 'Undocumented'. The 'Response headers' section shows 'date: Sun, 20 Sep 2026 21:56:34 GMT' and 'server: Kestrel'.

```
curl -X 'DELETE' \
'http://localhost:5269/api/Alugueis/3' \
-H 'accept: */*' 
```

Request URL

http://localhost:5269/api/Alugueis/3

Server response

Code	Details
204	Undocumented

Response headers

```
date: Sun, 20 Sep 2026 21:56:34 GMT
server: Kestrel
```

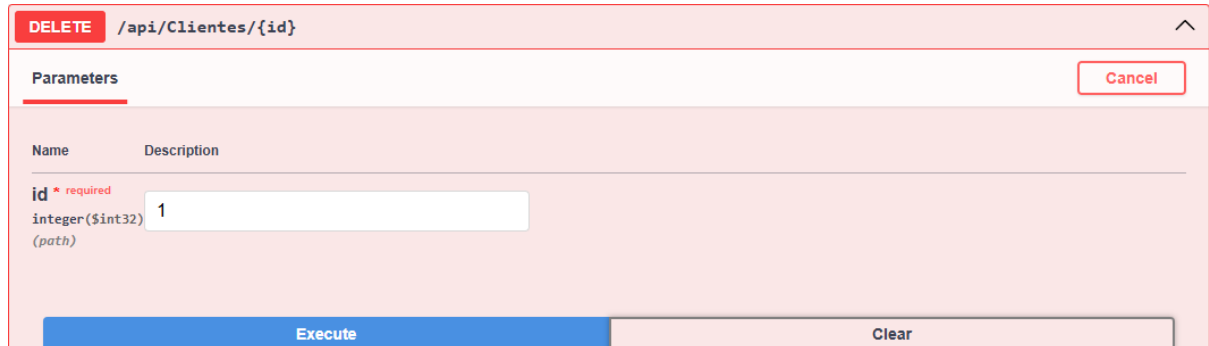
Figura 85 - Retorno do teste 42

Teste 43 - Erro: excluir cliente que possui aluguéis

Chamada: DELETE /api/Clientes/{id} (id = 1)

Resultado esperado: Código 400, porque a cliente possui aluguéis registrados.

Resultado obtido: Código 400, conforme o esperado.



The screenshot shows a REST client interface with a red header bar. The method is 'DELETE' and the URL is '/api/Clientes/{id}'. Below the header, there is a 'Parameters' section with a table. The table has two columns: 'Name' and 'Description'. There is one parameter named 'id' with a description 'integer(\$int32) (path)'. The value '1' is entered in the input field next to the parameter name. At the bottom, there are two buttons: 'Execute' (blue) and 'Clear' (light gray). A 'Cancel' button is also visible in the top right corner of the parameters section.

Name	Description
id * required	integer(\$int32) (path)

Figura 86 - Chamada do teste 43



The screenshot shows the response of the DELETE request. The 'Curl' section displays the command: `curl -X 'DELETE' \ 'http://localhost:5269/api/Clientes/1' \ -H 'accept: */*'`. The 'Request URL' section shows 'http://localhost:5269/api/Clientes/1'. The 'Server response' section shows a 'Code' of '400' and 'Details' of 'Error: Bad Request'. The 'Response body' section shows the message: 'Não é possível excluir um cliente que possui aluguéis registrados.' with a 'Download' button. The 'Response headers' section shows: 'content-type: text/plain; charset=utf-8', 'date: Sun,20 Sep 2026 21:56:36 GMT', 'server: Kestrel', and 'transfer-encoding: chunked'.

```
curl -X 'DELETE' \
'http://localhost:5269/api/Clientes/1' \
-H 'accept: */*'

Request URL
http://localhost:5269/api/Clientes/1

Server response
Code    Details
400     Error: Bad Request
Undocumented

Response body
Não é possível excluir um cliente que possui aluguéis registrados.

Response headers
content-type: text/plain; charset=utf-8
date: Sun,20 Sep 2026 21:56:36 GMT
server: Kestrel
transfer-encoding: chunked
```

Figura 87 - Retorno do teste 43

9. Vídeo de apresentação

O vídeo de apresentação do sistema (pitch) mostra o modelo do banco de dados, o código do projeto e a demonstração das APIs no Swagger. Ele está disponível no YouTube no link abaixo:

<https://www.youtube.com/watch?v=tAbzeDH3HnM>

10. Conclusão

Os 43 testes tiveram o resultado esperado. As operações de cadastro, consulta, atualização e exclusão funcionaram em todas as entidades, os cinco filtros retornaram os dados corretos e as validações bloquearam os dados inválidos com os códigos 400, 404 e 409.